MATHS4000
An Introduction to Lean Using Elementary Number Theory

Luke Smith — 10899560 Supervised by Martin Orr

Abstract.

The Lean proof assistant is a powerful tool that can verify the robustness of any mathematical result provided the appropriate translation into Lean code of the result's proof is supplied. We introduce the concept of a proof assistant and explain the motivation for the use of Lean over other options. We then formalise various results in elementary number theory, specifically in divisibility theory, and relating to binomial coefficients, prime numbers and congruences, with the intent of presenting these formalisations in a way that is understandable to beginners of Lean.

Contents

1 An Introduction to Lean **1**
- 1.1 An Introduction to Proof Assistants . . . 1
- 1.2 The Lean Proof Assistant . . . 2
- 1.3 Aims and Objectives . . . 5
- 1.4 Project Summary . . . 7

2 Divisibility Theory **9**
- 2.1 Introduction to Divisibility . . . 9
- 2.2 Divisibility of Finite Linear Combinations . . . 18
- 2.3 Greatest Common Divisor . . . 24
- 2.4 Bézout's Lemma . . . 34

3 The Binomial Theorem **42**
- 3.1 The Binomial Coefficient . . . 42
- 3.2 Pascal's Rule . . . 45
- 3.3 The Binomial Theorem . . . 48

4 Prime Numbers **56**
- 4.1 Introduction to Prime Numbers . . . 56
- 4.2 Euclid's Theorem . . . 61
- 4.3 Fundamental Theorem of Arithmetic . . . 65

5 Congruences **76**
- 5.1 Introduction to Congruences . . . 76
- 5.2 Fermat's Little Theorem . . . 79

Appendix **86**

Chapter 1

An Introduction to Lean

Mathematics has been the key driving force behind the creation and development of computers since their inception. For many years mathematicians have had the option to perform almost any calculation on a computer, and the vast majority possess, at minimum, the basic level of knowledge required to take advantage of this. However, most are unable to take advantage of a computer's ability to work with the theoretical side of mathematics, and in particular, do not benefit from proof assistants.

1.1 An Introduction to Proof Assistants

A proof assistant (or interactive theorem prover) is a tool, typically comprised of a programming language together with a proof verification system, that allows the user to formalise mathematical theory, and thus, through proper use, it can be shown without any doubt that a given result is indeed correct. This is a major benefit to the mathematical world, as it ensures the accuracy of future mathematical development. Moreover, proof assistants require the prover to explicitly state and prove all the mathematics that they have used to come to their conclusions, hence leading to an extremely rigorous representation of their proof as well as a more accessible piece of mathematics.

Naturally this leads us to wonder why proof assistants are not more popular. Truthfully, there are several barriers with regard to proof assistants becoming commonplace in mathematical research. The learning curve is steep, and thus it may take a significant amount of time to learn the language of a proof assistant to a high enough level that it can be useful for ones own research, which is often hard to justify. Furthermore, the considerable length of time spent formalising a proof that one regards as certain to be true may feel

seemingly pointless, especially considering that written (i.e. pen-and-paper) proofs are often able to skip over tedious but logically sound steps, unlike a formalisation, which requires significant detail. Additionally, the current human-led process of reviewing new mathematical breakthroughs for errors is refined to the extent that it is extremely effective, ostensibly negating the necessity for proof assistants. Despite all this, proof assistants are still quite a recent development, and with time they will become much more accessible and much more powerful. One can expect, especially considering the major advancements made with artificial intelligence in recent years, that proof assistants are the future of mathematical proof.

The first proof assistant was conceived by Nicolaas Govert de Bruijn in 1967, who began development on Automath, a system to verify mathematics [1, pp. 8–10]. In the present day, there are many options when it comes to choosing a proof assistant. However, according to a list consisting of 100 of the most famous theorems in mathematics, only four proof assistants have successfully formalised over 75% of them: Isabelle, HOL Light, Coq and Lean [2]. In this paper, we take advantage of Lean.

1.2 The Lean Proof Assistant

Lean was first launched in 2013 by Leonardo de Moura [3], and has undergone four iterations (with the most recent, Lean 4, released in 2021 [4]). When formalising mathematics in Lean, we write lines of code adhering to a specialised language. Each line is comprised of 'tactics' that represent the various implicit actions that one performs throughout proving a result by hand. Most proofs begin with various 'hypotheses' which are the starting assumptions that we work from to reach the final 'goal', the expression we are required to arrive at to proclaim the proof as complete.

As an initial overview of Lean, we formalise a very simple result: If a and b are integers, and $a + b = 1$, then $b + a = 1$. Obviously if we were to write a proof of this result manually, we would argue that addition is commutative over the integers, therefore if $a + b = 1$ then $b + a = 1$, which is exactly our 'goal'. This is implemented in Lean as follows:

```
theorem comm_eq_one (a b : ℤ) (h₁ : a + b = 1) : b + a = 1 := by
  rw[add_comm] at h₁
  exact h₁
```

The first line of a Lean formalisation is always a detailed declaration of the definition or proposition. We state our intention to begin a proof with the '`theorem`' label. Alternatively, we might choose to write '`lemma`' if it makes sense to do this in the context of the result, or for definitions, '`def`'. Next we decide on a name for our result — this can be anything, as long as it has not been used elsewhere. The formalisations in this report try to follow Lean naming conventions, although this is not a strict rule. This is followed by a declaration of variables and the family they belong to. Note that there can be more than one family of variables, for example we could define a to be an integer and b to be a natural number, in which case we write: '`(a :` $\mathbb{Z}$`) (b :` $\mathbb{N}$`)`' (be aware that when we refer to natural numbers in this paper, we include zero, as per Lean's convention). Subsequently we declare any hypotheses. Again more than one is possible, with the format being similar to that of the declaration of variables. The convention in this paper is to name hypotheses as '$\mathtt{h_i}$' where i is the i-th hypothesis, although this rule is sometimes not followed for particularly important hypotheses, or in exceptionally long formalisations where it can be hard to keep track of them all. Another convention we use with hypotheses is that we write the antecedent of any implication statements as a hypothesis, rather than as part of the main 'goal'. For example, if we formalise a proposition ordinarily written as $P \implies Q$, P will be declared as a hypothesis, and the goal will simply be Q, as opposed to setting the goal to be $P \implies Q$ with no hypothesis P. This is for the sake of consistency, but the latter approach is equally valid.

Following the declaration of hypotheses, we place a colon. This signals the beginning of the expression defining the 'goal' of the formalisation. We write our expression and finish with '`:= by`' to complete the declaration of the result, and begin the proof proper. The remainder of the code will be indented.

This project has been completed in Visual Studio Code. The Lean 4 extension required to code in Lean includes the Lean Infoview, a useful window that supplies the current state of the formalisation at the specified location. In our example, following '`:= by`' it displays the following:

```
1 goal
a b : ℤ
h₁ : a + b = 1
⊢ b + a = 1
```

It states that there is 1 goal, we have two integers a and b, a hypothesis $a + b = 1$ and a goal, that $b + a = 1$. The proof itself requires two lines, and

exactly two tactics, 'rw' and 'exact'. It is left as an exercise for the reader to interpret the meanings of these tactics — they will be explained in detail in Chapter 2. For reference, the following are the states of the Lean Infoview after the first and second lines of the proof proper.

```
1 goal
a b : ℤ
h₁ : b + a = 1
⊢ b + a = 1
```

```
No goals
```

Also observe the 'add_comm' — this is not a tactic, but rather a result. This is built into the 'Mathlib' library, an expansive library of mathematical results formalised in Lean and easily accessible for use in any Lean project — hence giving the primary motivation (combined with Lean's suitability for beginners and general ease of use) for our preference for Lean as opposed to the other 'main' proof assistants. Mathlib is an ongoing project, constructed through the collaboration of many mathematicians, but despite its infancy, it contains a significant number of definitions and theorems and, in particular, the majority of undergraduate mathematics, which is crucial to this project. We refer to [5] for a more in-depth (albeit partially outdated — note references to Lean 3) analysis of Mathlib. The mathematics formalised in Mathlib can be researched in various ways, although none are perfect. This includes Loogle [6] (best for when we know the name of the result), Moogle [7] (for when we know what the result 'looks like'), and via applications of tactics such as 'apply?' and 'rw?' (these relate to the tactics apply and rw, which we will encounter in later chapters).

It is important to define the boundaries regarding use of Mathlib in this project. Mathlib is necessary as it would be incredibly inefficient to formalise the many Mathlib results we take advantage of, but of course relying too heavily on Mathlib would not make for a meaningful project. As such we aim to employ definitions and theorems from Mathlib only when they are outside the scope of the project, including elementary mathematics (for example the commutativity of addition for integers), and results unrelated to number theory or the topic of discussion at a given time. However, this is not a strict rule, and generally the approach to Mathlib is quite pragmatic.

Typically, each formalisation in this project does not actually establish any

variables, contrary to what is demonstrated in the above example. Instead, we declare variables globally by establishing the beginning and end of a section of code relying on these variables with '`section`' and '`end`' respectively, and writing '`variable (...)`' to declare the identifier and type of all the variables we plan to use. This is to prevent the constant repetition that would present itself by declaring variables in each formalisation. Also note that we often 'implicitly' declare variables by using curly brackets (e.g. `variable {...}`) to remove the necessity of pairing variables with results — an explanation that will become more intelligible on achieving a higher familiarity with Lean. Furthermore, we begin each file with '`namespace my`' which declares that each result have '`my.`' positioned at the front of the name of the result (for example, '`my.comm_eq_one`'). By doing so we prevent any names from clashing with Mathlib results.

1.3 Aims and Objectives

There are two aims for this project:

- To formalise a selection of mathematical results using the Lean proof assistant.
- To produce a report on these formalisations that is comprehensible (and serves as an accessible introduction) for Lean beginners.

We discuss each aim individually, although there is some dependence between them. We begin with the first aim. Due to Lean's relatively active community, a significant number of mathematical results (and in particular, undergraduate mathematics) have already been formalised in the language. Any remaining results are outside the limits of someone with little experience, such as myself. In fact, one can view [8] to access a list of undergraduate mathematics yet to be formalised — the list is quite small, and although on paper some may seem manageable, it is important to note that the complexity of a result is not necessarily correlated to how well it might translate into the language of a proof assistant! Consequently, we are forced to formalise results for which there already exist formalisations. Nevertheless, the official code for most results is generally not comprehensible for a Lean novice, thus the work is meaningful in that we supply more user-friendly formalisations, and of course it provides myself with the experience of working with Lean. Furthermore, there are many unique approaches to formalising any given result (even if only one written proof exists), and as such it is likely that some of my own formalisations significantly differ from pre-existing formalisations

(and of course, some may be similar). As a note, every formalisation is my own, unless it is made explicitly clear that it is based on the work of others.

It is important that the selection of mathematical results to be formalised is coherent, and that they are easily understandable or have been encountered before by the intended audience. We require a choice of topic that allows focus to remain solely on the code, and not on the theoretical side. As such we base the formalisations in this project on certain aspects of elementary number theory, and in particular, ideas pertaining to divisibility theory, prime numbers and congruence. Knowledge of these topics is assumed but definitions, results and accompanying proofs will be declared at the appropriate time, although significant detail may be omitted. Note that none of the mathematical theory, including definitions, results, and proofs, used in this paper are my own — the provided proofs may vary slightly from the traditional approach (this will be explained in the next section) but they are always derived from said approach. Generally the relevant theory in this paper can be found in any elementary number theory textbook, including, but not limited to, [9], [10] and [11]. The selection of results themselves, as well as the organisation of the results, has been revised numerous times over the course of writing this paper, but initial plans were heavily influenced by [11] and [12].

The key challenge to fulfilling the first aim is of course knowledge of Lean. As mentioned earlier, Lean is still new to the mathematical world, and especially Lean 4, the version of Lean adopted in this project. Hence, learning resources are lacking, although some excellent material still exists, with a significant portion of my studies spent working with a very detailed interactive textbook ([13]), after an initial introduction to Lean via the Natural Number Game ([14]).

The importance of the second aim is significantly elevated by the previous paragraph. This report provides an introduction to the concept of a proof assistant, as well as an introduction to Lean, including what it is and why we have chosen to use it over other proof assistants. We have already explored the overall structure of a Lean formalisation, and will discuss the various tactics and techniques involved in formalising any mathematical result as we encounter them in our own formalisations. With regard to the code, the formalisations themselves are often not written in the most efficient way possible, partially for the sake of readability, but also to make them easier to decipher for beginners. This differs from a traditional proof one can find in Mathlib, in which each formalisation is written with maximum efficiency in

mind, and as such is severely lacking readability.

1.4 Project Summary

Outside of this introductory chapter, this report contains four chapters and an Appendix. The overall structure of each chapter is as follows: firstly a lemma/theorem is stated, followed by a written proof — this proof is crafted to follow the Lean formalisation as best as possible, and so there may be some differences or quirks that may not be seen in the canonical proof. After this we begin a breakdown of the formalisation of the proof, complete with code listings at appropriate intervals. The code listings themselves are indented as seen in the code proper. Note that due to the significance of the Lean Infoview in following a formalisation, it may sometimes be difficult to realise the impact of each line of code when exclusively reading the text, and as such it is highly recommended to supplement reading with occasional examinations of the code (see [15]).

To close this introduction we offer brief descriptions of each chapter. Chapter 2 is the largest chapter — covering divisibility theory, including divisors, the greatest common divisor and concluding with Bézout's Lemma. This chapter serves as an in-depth introduction to Lean, offering detailed explanations of many of the tactics that are to be used throughout this project.

Chapter 3 covers results relating to the binomial coefficient, including Pascal's rule and the Binomial Theorem. This chapter continues to introduce some new tactics, including an especially useful tactic '`calc`', which allows for a particularly smooth reading experience. Of course, the binomial coefficient and related results are not strictly number theory, but they do allow for some excellent formalisations, and further, the results themselves are required in Chapter 4 and Chapter 5.

Chapter 4 looks at prime numbers, including Euclid's Lemma, Euclid's Theorem and ending with the Fundamental Theorem of Arithmetic. At this stage most tactics have been introduced, although there are still plenty of new ideas. The Fundamental Theorem of Arithmetic is a particularly large formalisation, and provides plenty of insight.

Chapter 5 is the final chapter, covering congruences. It is predominantly occupied by an investigation of Fermat's Little Theorem, which takes results and ideas from each of the previous chapters, thus serving as a stellar con-

cluding formalisation.

The Appendix mainly contains formalisations that are used as part of the formalisation of a result in the main text, but have not been discussed themselves. They have been left out due to organisational reasons, but are still relevant and may be of interest to the reader. The Appendix also contains some formalisations that are not used anywhere in this paper, but are relevant to elementary number theory and may still be of interest. Note that no written proofs or code explanations are provided.

Chapter 2

Divisibility Theory

In this chapter we formalise some fundamental results in divisibility theory, introducing the reader to a wide variety of tactics required to formalise any result in Lean.

2.1 Introduction to Divisibility

We begin with some simple properties relating to divisors. The idea is to begin familiarising the reader with the Lean theorem prover, while simultaneously establishing a foundation of results that will aid future proofs. Not every major property will be covered, only those that will be required later, or those that provide particularly illustrative examples. Note that each result stated in this section will be grouped into the same section of Lean code (i.e. inside a `section` ... `end` pair), with variables (implicitly) defined to be elements of a commutative semiring. We will describe these elements as being integers — we use a commutative semiring to allow the formalised results to be usable with both variables of 'integer' type and variables of 'natural number' type (Lean is strict with regard to the variable type; it does not view variables of 'natural number' type as being integers).

The reader should be familiar with the central idea of divisibility theory:

Definition 2.1. *Let a and b be integers. Then a divides b (denoted $a \mid b$) if there exists an integer c such that $b = ac$.*

Our first result is a critical, yet trivial, result in number theory.

Lemma 2.2. *Let a be an integer. Then a divides a.*

Proof. By Definition 2.1, if there exists an integer c such that $a = ac$ then $a \mid a$. Choosing $c = 1$ satisfies $a = ac$ and so $a \mid a$. □

We follow the syntax introduced in the first chapter. Firstly we choose an appropriate label to describe our result ('`lemma`') and decide on a name for our result ('`dvd_reflex`'). Then, after the colon, we state our proposition using purely mathematical notation (where '`\dvd`' is used to obtain the notation denoting divisibility), and write '`:= by`' to initiate the start of the proof.

```
lemma dvd_reflex : a | a := by
```

Due to the simplicity of the proof we require just two more lines of code to complete the formalisation. Recall from Chapter 1 that these lines are to be indented. Two tactics are employed, '`use`' and '`rw`'.

We detail the '`use`' tactic first. This tactic is one that is used sparingly throughout this report due to its limited applicability. It is utilised in situations where the goal contains an explicit or implicit existential quantifier — implicit referring to scenarios in which an existential quantifier directly follows from some definition or notation in the goal. One such scenario can be found in our formalisation — the goal is set to 'a divides a'. This implies that there exists some integer such that, multiplied to a, we arrive at a. We provide '`use`' with a witness, 1, which it then substitutes into the goal, thus removing the existential quantifier and leaving us with a simple goal, that a is equal to a multiplied by 1.

```
    use 1
```

Although the remaining goal is quite obviously true to the user, due to the nature of Lean we do not have a complete formalisation until both sides of the equation are exactly the same. This leads us to the second tactic of this formalisation, the '`rw`' tactic. This is the tactic most frequently used throughout this report, and is arguably the most important that Lean has to offer. It is short for 'rewrite', and paired with some equality (who's name is written inside of square brackets immediately after `rw`), it substitutes into the goal the right-hand side of the equality, in place of the left-hand side. It does this at all possible instances, but is not recursive. It can also take multiple results at once, performing the rewrite sequentially until either all results have been applied or it reaches a result that cannot be applied. Keep in mind that this tactic is directional. If we want to substitute the left-hand side of an equality into the goal in place of the right-hand side, it is required

to input a left-facing arrow (written '`\l`' in Lean) before stating the name of the equality. Additionally, note that '`rw`' not only performs a substitution, but then proceeds to check if both sides of the equality are definitionally equivalent, and if so, closes the goal, thus making it a slightly more powerful version of the '`rewrite`' tactic it is based on. We choose to use the more powerful tactic at all times in this report, as the cost of checking for reflexivity is largely negligible, thus making it more convenient.

Returning to our formalisation, we pair `rw` with '`mul_one`', a Mathlib result stating that $a \cdot 1 = a$. As described in the previous paragraph, the right-hand side (a) replaces occurrences of the left-hand side ($a \cdot 1$) in the goal, thus closing the goal, as we have that $a = a$.

```
    rw[mul_one]
```

It is important to note that, if the goal was $a = 1 \cdot a$, attempting a rewrite using `mul_one` would raise an error. It would first be necessary to swap around 1 and a using commutativity of multiplication, `mul_comm`, and only then could `mul_one` be used. This often leads to two iterations of a result being defined for the sake of convenience, a rule that we take advantage of in the following lemma..

Lemma 2.3. *Let a and b be integers. Then a divides ab.*

Proof. By Definition 2.1, if there exists an integer c such that $ab = ac$ then $a \mid ab$. Choosing $c = b$ satisfies $ab = ac$ and so $a \mid ab$. □

The formalisation that a divides ab is again very simple (in fact, more simple than Lemma 2.2), requiring the application of the `use` tactic paired with b. This yields that $ab = ab$. Note that, similar to `rw`, `use` is able to detect reflexivity and close the goal.

```
lemma dvd_mul_right : a | a * b := by
  use b
```

This result is not enough if we are dealing with a dividing ba, so we devise a separate formalisation. To formalise that a divides ba, we are initially forced to use the `rw` tactic together with `mul_comm`, to arrive at a new goal: $a \mid a \cdot b$.

```
lemma dvd_mul_left : a | b * a := by
  rw[mul_comm]
```

Notice that the goal is now equivalent to `dvd_mul_right`. There are two possible approaches from here to close the goal. The first approach is to type `use b`, as previously demonstrated. This is a perfectly valid approach — however, for the purposes of achieving a greater understanding of Lean, we can instead reason that since the goal is exactly a goal that we have already proven, we can utilise the '`exact`' tactic.

The `exact` tactic is easily understood. It is adopted whenever the goal is precisely a result that we know to be true. This can either be any result formalised in Lean, or any hypothesis. Pairing the tactic with such a result or hypothesis closes the goal. Our goal is currently that $a \mid a \cdot b$, which is exactly `dvd_mul_right` (or '`my.dvd_mul_right`' due to the namespace), and so pairing this with the `exact` tactic completes the formalisation.

```
exact my.dvd_mul_right
```

We use this formalisation as an opportunity to better explain the idea of implicit variables. In the above example, a and b are defined implicitly, that is, at the top of our file we have '`variable {a b : R}`' where R is an arbitrary commutative semiring. If we were to instead define a and b explicitly, i.e. '`variable (a b : R)`', then we would be forced to explicitly provide a and b as the variables we are working with when calling the `exact` tactic. That is, we would write `exact my.dvd_mul_right a b`. This can occasionally be advantageous, but in general it is more of an inconvenience, and as such we typically avoid defining explicit variables.

Next we formalise a generalisation of Lemma 2.3.

Lemma 2.4. *Let a and b be integers and suppose a divides b. Then a divides bx for all integers x.*

Proof. Suppose $a \mid b$. Then there exists an integer c such that $b = ac$. So $bx = (ac)x = a(cx)$. By Lemma 2.3 a divides $a(cx)$, hence a divides bx. □

We begin as usual by writing the first line of the formalisation, which outlines what we intend to prove. However, this time we include a hypothesis $\mathtt{h}_1$, which is that a divides b.

```
lemma dvd_mul_right_general (h₁ : a ∣ b) : a ∣ b * x := by
```

The formalisation of this result requires a new tactic, '`rcases`'. This tactic reduces a given hypothesis into multiple, more focused hypotheses. It

demands a hypothesis to unpack, and at least two new identifiers that the hypothesis can be unpacked into, which we provide inside of angle brackets (\<, \> in Lean). We have the hypothesis $\mathtt{h_1}$, that a divides b. This has the implication that there is some integer who's product with a is equal to b. Applying the `rcases` tactic on this hypothesis with provided identifiers c and $\mathtt{h_1}$ defines c as a new variable of the correct type (an element of a commutative semiring) and $\mathtt{h_1}$ as a new hypothesis: that $b = a \cdot c$. Similarity to the `use` tactic may be observed. The `rcases` tactic performs a similar function to the `use` tactic in this instance, however, there is one major difference. The `rcases` tactic is able to define previously undefined variables, whereas `use` can only take advantage of existing variables. Furthermore, the primary function of the `use` tactic is to reduce the goal — it does not work on hypotheses. Note that `rcases` typically does not perform a similar function to `use`, this is just a specific case in which it does.

```
rcases h₁ with ⟨c, h₁⟩
```

To clarify our choice of identifier for the new hypothesis: we choose to use the same identifier as our original hypothesis as the original hypothesis is annihilated on interaction with `rcases`.

The rest of the formalisation is rather straightforward. We first draw on the `rw` tactic to substitute $\mathtt{h_1}$ into the goal, giving an updated goal: that a divides $a \cdot c \cdot x$. Note that in Lean, acx refers to $(ac)x$. Using associativity of multiplication, given as `mul_assoc` in Mathlib, we can again take advantage of `rw` to replace acx with $a(cx)$. Note that we do not require a separate use of the `rw` tactic to do this, we can simply separate our results by a comma inside of the square brackets.

```
rw[h₁, mul_assoc]
```

All that remains is to prove that a divides $a \cdot (c \cdot x)$. This is exactly Lemma 2.3.

```
exact my.dvd_mul_right
```

Once more we want to have available to us the version of this result with commutativity of multiplication applied, that is, if a divides b then a divides xb. Analogous to Lemma 2.3, we can simply use `rw` with `mul_comm` and the `exact` tactic. There is one minor difference however. As a consequence of the defining of a hypothesis in the statement of the lemma in Lean, we are forced

to provide with the `exact` tactic (together with the formalised result that it is equivalent to) evidence that the hypothesis holds for the new lemma. This is trivial for this result, as we again define the hypothesis $\mathtt{h_1}$, that a divides b.

```
lemma dvd_mul_left_general (h₁ : a ∣ b) : a ∣ x * b := by
  rw[mul_comm]
  exact my.dvd_mul_right_general h₁
```

Next we consider a lemma that can be split into two cases, both of which can be formalised in a similar way as has just been demonstrated, but which combine to form a more significant result.

Lemma 2.5. *Let a, b and c be integers and suppose ab divides c. Then a divides c and b divides c.*

Proof. Suppose $ab \mid c$. We first check if $a \mid c$. By Definition 2.1 there exists an integer, say x, such that $c = (ab)x$. We have $c = (ab)x = a(bx)$. By Lemma 2.3, $a \mid a(bx)$ and so $a \mid c$.

Now we check if $b \mid c$. Since $ab \mid c$, $ba \mid c$, therefore by the same argument as above we have $b \mid c$. □

The aim is to formalise that both a divides c and b divides c in one Lean result, but for organisational purposes and convenience for later proofs we first establish two standalone results. We neglect to analyse these in detail as they are almost identical to the formalisations of Lemma 2.4.

```
lemma prod_left_dvd (h₁ : a * b ∣ c) : a ∣ c := by
  rcases h₁ with ⟨x, h₁⟩
  rw[h₁, mul_assoc]
  exact my.dvd_mul_right

lemma prod_right_dvd (h₁ : a * b ∣ c) : b ∣ c := by
  rw[mul_comm] at h₁
  exact my.prod_left_dvd h₁
```

It remains to combine these results. The theorem statement is uncomplicated, we simply combine the two results that we desire with an and symbol (`\and` in Lean).

```
lemma prod_left_right_dvd (h₁ : a * b ∣ c) : a ∣ c ∧ b ∣ c := by
```

It should be obvious that the `exact` tactic along with the two results just provided will come into play in formalising this result. The only real difficulty comes from handling the conjunction. There are two primary approaches to this. The most efficient approach is to employ the `exact` tactic, paired with a set of angle brackets containing both results.

```
exact ⟨my.prod_left_dvd h₁, my.prod_right_dvd h₁⟩
```

This closes the goal in one simple line, and for situations in which we have already proven results for both the left and right side of the goal (split by a conjunction) this is recommended. However, this will usually not be the case, which necessitates a different approach to handling the conjunction. This is where the `constructor` tactic becomes beneficial. This tactic has the main purpose of breaking down conjunctions and biconditionals. In this case, the goal is split into two separate subgoals: firstly that a divides c, and then that b divides c. Proving both of these cases results in a complete formalisation, which is of course trivial using `exact` and our existing formalisations.

```
constructor
· exact my.prod_left_dvd h₁
· exact my.prod_right_dvd h₁
```

Note that we have some 'bullet points' inside of our Lean proof. These define the beginning of the proof of each case. They are not strictly required, but are recommended. When the goal is split into multiple cases, it is possible to continue as normal, proving each case sequentially, and when one case is proven, it closes and allows access to the next case. There are two drawbacks to working in this manner. Firstly, it is forced upon the user to work on each case successively, with no option to move onto the next case until the current case has been proven. Secondly, larger proofs become much harder to read and follow. Using the bullet points allows for unrestricted access to all cases at any time, and makes the code much easier on the eye.

We finish this section with two fundamental results.

Theorem 2.6. *Let a, b, and c be integers. If a divides b and b divides c then a divides c.*

Proof. Suppose $a \mid b$ and $b \mid c$. Then there exists integers x and y such that $b = ax$ and $c = by$. We have $c = by = (ax)y = a(xy)$, and so by Lemma 2.3, $a \mid c$. □

The formalisation is rather simple, only requiring tactics we have encountered before. The `rcases` tactic is used twice to restate both hypotheses to their explicit form. From there they can be substituted into the goal, and by multiplicative associativity we obtain a new goal: a divides $a(xy)$,which is exactly Lemma 2.3.

```
theorem dvd_trans (h₁ : a ∣ b) (h₂ : b ∣ c) :  a ∣ c := by
  rcases h₁ with ⟨x, h₁⟩
  rcases h₂ with ⟨y, h₂⟩
  rw[h₂, h₁, mul_assoc]
  exact my.dvd_mul_right
```

Theorem 2.7. *Let a, b, c and d be integers. If a divides b and c divides d. Then ac divides bd.*

Proof. Suppose $a \mid b$ and $c \mid d$. Then there exists integers x and y such that $b = ax$ and $d = cy$. We have $bd = (ax)(cy) = a(xc)y = a(cx)y = (ac)(xy)$, and so by Lemma 2.3, $ac \mid bd$. □

Again the formalisation of this theorem is elementary, although it does require some laborious algebraic manipulation, which forces the deployment of a new tactic. As previous, we first use `rcases` twice to restate the hypotheses, and then substitute the hypotheses into the goal. It remains to prove that ac divides $ax(cy)$. We aim to manipulate this into the form $ac(xy)$, so that we can conclude the proof with Lemma 2.3. We begin by using the `rw` tactic paired with `mul_assoc`. This is to adjust $ax(cy)$ to be $axcy$ (i.e. $(ax)cy$). Notice that $ax(cy)$ aligns with the right-hand side of the equality defining `mul_assoc`, rather than the left-hand side. Recall that this requires the use of a left-facing arrow before typing the name of the result.

```
theorem mul_dvd_mul (h₁ : a ∣ b) (h₂ : c ∣ d) : a * c ∣ b * d := by
  rcases h₁ with ⟨x, h₁⟩
  rcases h₂ with ⟨y, h₂⟩
  rw[h₁, h₂, ← mul_assoc]
```

The next step is to isolate xc. This is achieved by again utilising `mul_assoc`, this time in the forward direction. One can observe that simply writing `rw` paired with `mul_assoc` will just undo our previous application of it. Instead we use a new tactic, '`nth_rw`'. This tactic performs exactly the function of `rw`, except only at a specified occurrence. We find that the second case of `mul_assoc` gives $a(xc)y$. Consequently we pair `nth_rw` with 2, and as is customary for the `rw` tactic, write `mul_assoc` in square brackets beside it.

```
  nth_rw 2 [mul_assoc]
```

Continuing, we aim to swap xc to cx, which clearly requires multiplicative commutativity, given as '`mul_comm`' in Mathlib, which to be specific, is that $ab = ba$ for some a and b. Analysing the expression $a(xc)y$, it can be concluded that we need the fourth instance of commutativity of multiplication to get the desired expression (often it is easier to apply a trial-and-error approach in Lean rather than evaluating the expression for the intended case). Once commutativity is applied in the correct place, it is straightforward to reach the expression $ac(xy)$ using multiplicative associativity, and to conclude the formalisation we invoke Lemma 2.3.

```
  nth_rw 4 [mul_comm]
  rw[← mul_assoc, mul_assoc]
  exact my.dvd_mul_right
```

A much more convenient approach to formalising the above result does exist, but due to its triviality, does not align with the goals of this chapter, hence the emphasis on the more convoluted approach involving algebraic manipulation. This approach is as follows: we forgo performing a considerable portion of the algebraic manipulation ourselves entirely, using the '`ring_nf`' tactic. Given an expression has variables contained in a commutative ring (or commutative semiring), `ring_nf` simplifies this expression to a canonical form using the ring axioms. For example, through `ring_nf` the expression $ax(cy)$ would generally become $axcy$. However, in our formalisation the goal is to prove that ac divides $ax(cy)$, so `ring_nf` recognises this and the canonical form is instead set to be $acxy$. From here it is a simple application of multiplicative associativity and Lemma 2.3.

```
theorem mul_dvd_mul (h₁ : a ∣ b) (h₂ : c ∣ d) : a * c ∣ b * d := by
  rcases h₁ with ⟨x, h₁⟩
  rcases h₂ with ⟨y, h₂⟩
  rw[h₁, h₂]
  ring_nf
  rw[mul_assoc]
  exact my.dvd_mul_right
```

2.2 Divisibility of Finite Linear Combinations

We will now formalise a handful of results concerning the divisibility of linear combinations, with the ultimate aim of formalising that if some integer divides integers b_i for $i = 0, \ldots, n$, then this integer divides any finite linear combination of the b_i. We continue initially with variables defined in a commutative semiring

We begin our analysis with a simple case of the proposition:

Theorem 2.8. *Let a, b and c be integers. If a divides b and a divides c then a divides $bx + cy$ for all integers x and y.*

Proof. Suppose $a \mid b$ and $a \mid c$. Then there exists integers s and t such that $b = as$ and $c = at$. Then for integers x and y we have $bx + cy = asx + aty = a(sx + ty)$, and so by Lemma 2.3, $a \mid bx + cy$. □

We open with an application of `rcases` on both hypotheses to obtain their explicit forms, and substitute the equations we acquire into the goal with `rw`.

```
theorem dvd_add_of_two_mul (h₁ : a ∣ b) (h₂ : a ∣ c) : a ∣ b * x + c
    * y := by
  rcases h₁ with ⟨s, h₁⟩
  rcases h₂ with ⟨t, h₂⟩
  rw[h₁, h₂]
```

It remains to prove that a divides $asx + aty$. We accomplish this using the Mathlib result '`mul_add`', which proves that $a(b + c) = ab + ac$ for some a, b and c. Notice that the reverse direction will be needed when applying this to our expression. Further, notice that it is compulsory to isolate sx and ty before we can apply `mul_add` (as $asx+aty$ refers to $(as)x+(at)y$ and we need $a(sx)+a(ty)$) . This is achieved through the use of `rw` paired with `mul_assoc`; there is no reason to use `nth_rw` as both cases of multiplicative associativity need to be targeted. As the same tactic is essentially being repeated twice (that is, `rw[mul_assoc]`), it is viable to write '`repeat`' before `rw[mul_assoc]`. Using `repeat` causes the tactic it is paired with to continuously apply itself to the goal until it is no longer able to. It forces an error in scenarios in which the tactic would repeat forever, for example `rw[mul_comm]`. Note that using `repeat` is entirely optional, and an argument can be made that it may even be more efficient in this particular formalisation to simply write `mul_assoc` twice with the same `rw`.

```
  repeat rw[mul_assoc]
```

Now the goal is to deduce that a divides $a(sx) + a(ty)$, and so we can pair `← mul_add` with the `rw` tactic to get a new goal, that a divides $a(sx + ty)$, and thus we can close the goal using Lemma 2.3.

```
  rw[← mul_add]
  exact my.dvd_mul_right
```

It is clear from this result that if a divides b and a divides c, then a divides $b + c$. For practicality, we create a separate result in Lean for this form, as we can expect it to be useful later.

Corollary 2.9. *Let a, b and c be integers. If a divides b and a divides c then a divides $b + c$.*

Proof. Follows from Theorem 2.8 with 1 as the coefficients. □

Obviously to formalise this corollary, it is simply necessary to show that $b + c$ equals $b \cdot 1 + c \cdot 1$, and then we can apply Theorem 2.8. Firstly we capitalise on the Mathlib result `mul_one`, which proposes that $a \cdot 1 = a$ for some a. We pair `rw` with `← mul_one`, noting that `← mul_one` necessitates the specification of which term it needs to be applied to, in this case $b + c$. This simplifies the goal to proving that a divides $(b + c) \cdot 1$. and from here it is a simple case of using `mul_add`, and finally we conclude with Theorem 2.8.

```
lemma dvd_add (h₁ : a ∣ b) (h₂ : a ∣ c) : a ∣ (b + c) := by
  rw[← mul_one (b + c), add_mul]
  exact my.dvd_add_of_two_mul h₁ h₂
```

Having established the two-variable case for the divisibility of linear combinations, the next step is to formalise the general case. Rather than confronting this task all at once, we prove a smaller result that we can use to prove the larger result. Note that for the remainder of this section, we define n to be of natural number type.

Lemma 2.10. *Let a be an integer and suppose that a divides integers b_0, b_1, $\ldots$, b_n for some $n \geq 0$. Then a divides their total sum.*

Proof. We proceed by induction. Suppose that $n = 0$. If $a \mid b_0$, then clearly $a \mid \sum_{i=0}^{0} b_i = b_0$.

For the inductive hypothesis, assume that if $a \mid b_i$ for all $i = 0, \ldots, k$, then $a \mid \sum_{i=0}^{k} b_i$. We aim to show that for all $i = 0, \ldots, k+1$, $a \mid \sum_{i=0}^{k+1} b_i$. We have $\sum_{i=0}^{k+1} b_i = \left(\sum_{i=0}^{k} b_i\right) + b_{k+1}$. Therefore if $a \mid \sum_{i=0}^{k} b_i$ and $a \mid b_{k+1}$ then a will divide their sum by Corollary 2.9.

First we show that $a \mid \sum_{i=0}^{k} b_i$. By the inductive hypothesis this is true if $a \mid b_i$ for all $i = 0, \ldots, k$, which we know is true by our initial assumption that a divides integers b_0, b_1, $\ldots$, b_n with $n = k+1$. Note that this assumption also makes it obvious that $a \mid b_{k+1}$. Hence by Corollary 2.9 we are done. □

Note that there is a much easier proof not involving induction, but it does not translate pleasantly into Lean, as Lean does not interact well with finite numbers of elements. This will be discussed in the next few paragraphs.

This formalisation takes advantage of several new, but mostly simple tactics. It also introduces some new ideas. Firstly, Mathlib's 'Finset' library is used, which defines the concept of a finite set, and various facts about finite sets. The use of finite sets is a requirement in order to define the integers $b_0, \ldots, b_n$. Typically in a mathematical proof involving a set of $n+1$ integers, we may write 'Let $b_0, b_1, \ldots, b_n \in \mathbb{Z}$'. This cannot be translated easily into Lean. We employ a function $b : \mathbb{N} \to R$ which sends natural numbers to our commutative semiring. We then define a finite set $\{0, 1, \ldots, n\}$ (using `Finset.range (n+1)`). Now for any i in this set, we write `b i` to be the function $b(i) = b_i$, thus providing us with our integers $b_0, b_1, \ldots, b_n$.

```
lemma dvd_sum (h₁ : ∀ i ∈ Finset.range (n + 1), a ∣ b i) : a ∣ Σ i ∈
    Finset.range (n + 1), b i := by
```

Proceeding with the proof, we introduce a new tactic, `induction'`, which begins a proof by induction. We specify that we want to induct on n, and establish k and `ih` as the induction variable and inductive hypothesis respectively. It is possible to specify the type of induction being employed, but without any specifications (as seen in this example), weak induction is assumed. Recall that weak induction involves two stages: the base case, and the inductive step. The `induction'` tactic initialises these stages for us, similar to as seen from the `constructor` tactic (in Lemma 2.5).

```
  induction' n with k ih
```

It remains to prove each stage. We again use 'bullet points' to organise

the formalisation in a tidy manner. Starting with the base case, we take advantage of an invaluable tactic — `simp`. This tactic simplifies the target, leveraging any other hypotheses or results it has access to. The `simp` tactic is just one tactic that can accelerate the formalisation process, although it can come at the cost of transparency, hence why it has not been used thus far. Nevertheless, since the base case is simply an application of a few straightforward results, it seems frivolous to spend time manually simplifying, and thus it is left as an exercise to the reader. Note that it is specified to apply `simp 'at *'`. The asterisk specifies that we want to apply the tactic to everything — that is, the goal as well as all hypotheses. In this case, both the hypothesis $\mathtt{h}_1$ and the goal are simplified to `'a | b 0'`. Clearly the goal is equivalent to $\mathtt{h}_1$, so we can conclude that the base case is true.

```
· simp at *
  assumption
```

Note that `assumption` is employed rather than `exact` $\mathtt{h}_1$. Both methods close the goal, although `assumption` does not require the specification of a hypothesis to do so. It scours through each hypothesis and identifies the correct one for us. Of course, when there is only one hypothesis this is rather superfluous, but it can be convenient for larger formalisations.

Now we proceed with the inductive step. Due to our provisions when using the `induction'` tactic, `ih` is defined as the inductive hypothesis, and k is the induction variable. As given in the written proof, we first split the sum so that i reaches a maximum of k, with b_{k+1} separate. This can be achieved using the Mathlib result `'Finset.sum_range_succ'`. Next, we present a pivotal tactic for any proficient Lean user: `apply`. This tactic can be used on both hypotheses and goals, but the function is slightly different. When used on the goal, `apply` allows us to prove the goal, by proving a result that implies the goal. For example, suppose that for propositions P and Q, we have that P implies Q. If our current goal is Q, then if we prove P, then Q must follow, and so applying P implies Q to the goal causes the goal to be changed to proving P. This is how we exploit Corollary 2.9. If we can prove that a divides the sum of the elements of $\{b_0, \ldots, b_k\}$ and that a divides b_{k+1}, then it follows that a divides the sum of the elements of $\{b_0, \ldots, b_k\}$ together with b_{k+1}, which is exactly our goal. Note that `apply` will split the hypotheses required for the goal to follow into separate subgoals (in this case, we obtain two subgoals — that a divides the sum of the elements of $\{b_0, \ldots, b_k\}$ and that a divides b_{k+1}), allowing for one subgoal to be proven at a time — similar to what we have seen with the `constructor` and `induction'` tactics.

If we were to use `apply` on a hypothesis, it will have the opposite effect. If the hypothesis is Q, and we know that P implies Q, then applying P implies Q to the hypothesis Q clearly cannot be allowed as a hypothesis is some idea or expression that we know to be true, and it is not factual that P is true even if Q is true. On the other hand, if the hypothesis is P, then applying P implies Q to P will change the hypothesis to Q, as we know that since P is true, Q is also true.

```
· rw[Finset.sum_range_succ]
  apply my.dvd_add
```

The first subgoal is to prove that a divides the sum of the elements of $\{b_0, \ldots, b_k\}$. Here we can use the inductive hypothesis, which states that this is true if a divides each element of the set. Once more this is an opportunity to utilise `apply` (together with `ih`), thus changing the goal to be proving that a divides each element of the set.

```
· apply ih
```

It remains to prove that a does indeed divide each $b_0, \ldots, b_k$. Lean understands this as 'for all i in $\{0, 1, \ldots, k\}$, a divides b_i. This is obvious by $\mathtt{h_1}$ (with $n = k + 1$). Unfortunately, Lean does not see it this way, so it is necessary to deconstruct the current goal into something more palatable. We make use of the `intro` tactic, which dissects the goal by creating new hypotheses taken from the goal. Typically this is employed when the goal includes universal quantifiers and/or implications. In this instance we have a universal quantifier attached to i. Hence, typing `intro i` defines i as an arbitrary natural number, and removes the universal quantifier, modifying the goal into an implication statement: that if i is in $\{0, 1, \ldots, k\}$, then a divides b_i. Since we now have an implication statement, we can use `intro` again. We set the antecedent to be $\mathtt{h_2}$. Although we may write these two `intro` statements separately, it is more compact to write them as one.

```
  intro i h₂
```

The goal is now to simply show that a divides b_i. As a result of our use of `intro` we can use `apply` together with $\mathtt{h_1}$ to generate a new goal — that i is in $\{0, 1, \ldots, k + 1\}$ (since $\mathtt{h_1}$ is essentially an implication statement that if i is in $\{0, 1, \ldots, k + 1\}$, then a divides b_i). Again this is clear to the human reader, but Lean requires further simplification. We use the Mathlib result

'`Finset.mem_range`' which reduces i's membership in a set to an inequality. Rewriting both $\mathtt{h}_2$ and the goal, we get that $i < k + 1$ and $i < k + 1 + 1$ respectively. At this point it is possible to close the goal, and this can be achieved by using `linarith`. This is a tactic which essentially solves simple goals that involve linear arithmetic. It is similar to `simp` in that it allows us to skip some of the more menial work in a formalisation. It recognises that the goal $i < k+1+1$ is clearly proven by the hypothesis that $i < k+1$, and thus we have proven the first subgoal.

```
apply h₁
rw[Finset.mem_range] at h₂
rw[Finset.mem_range]
linarith
```

For the interested reader, if we decided against using `linarith`, we can actually close this goal in one line: `exact Nat.lt_add_right 1` $\mathtt{h}_2$, where the result '`Nat.lt_add_right`' is a Mathlib result stating that for natural numbers a, b, c, if $a < b$ then $a < b + c$. Although this closes the goal in just one line, this will not always be the case in situations where `linarith` is applicable, and furthermore, locating the result was not instant unlike the use of `linarith`.

The final step in this formalisation is to prove the second subgoal: that is, to prove that a divides b_{k+1}. The method applied is not too dissimilar to as seen in the first subgoal, although it is much shorter. Firstly, we use `apply` together with $\mathtt{h}_1$, which creates a new goal, that $k + 1$ is an element of $\{0, \ldots, k + 1\}$. Rewriting the goal with `Finset.mem_range` then leaves $k + 1 < k + 1 + 1$, which is clearly true, and `linarith` agrees with that assessment.

```
· apply h₁
  rw[Finset.mem_range]
  linarith
```

With the formalisation of Lemma 2.10 complete, the final theorem of this subsection follows rather trivially.

Theorem 2.11. *Let a be an integer and suppose that a divides integers b_0, b_1, ..., b_n. Then a divides every integer linear combination of them.*

Proof. Suppose that $a \mid b_i$ for all $i = 0, \ldots, n$. We aim to prove that $a \mid \sum_{i=0}^{n} c_i b_i$ for integers c_i. By Lemma 2.10, this follows if we prove that

$a \mid c_i b_i$ for all $i = 0, \ldots, n$. By Lemma 2.4 we can instead prove that $a \mid b_i$ for all $i = 0, \ldots, n$. This is true by assumption. □

This formalisation is perhaps anticlimactic as a result of prior work in this chapter. The setup is rather similar to the formalisation of Lemma 2.10, although we define an extra function c, defined in the exact same way as b.

```
theorem dvd_sum_linear_combination (h₁ : ∀ i ∈ Finset.range (n +
    1), a ∣ b i) : a ∣ Σ i ∈ Finset.range (n + 1), c i * b i := by
```

Predictably we utilise `apply` together with `dvd_sum` to reduce the goal to proving that for all i in $\{0, \ldots, n\}$, a divides $c_i b_i$. As seen previously, this calls for the `intro` tactic to remove the universal quantifier and implied conditional, leaving the goal as a divides $c_i b_i$. Then we once more use `apply` with Lemma 2.4. The goal reduces to a divides b_i which is implied by our initial assumption given that i is in $\{0, \ldots, n\}$, which is true by the hypothesis we obtained through using `intro`.

```
  apply my.dvd_sum
  intro i h₂
  apply my.dvd_mul_left_general
  apply h₁
  assumption
```

2.3 Greatest Common Divisor

For the remainder of this chapter we set all variables to have integer type.

Having established some fundamental properties of divisibility, naturally we turn our attention towards the greatest common divisor. This will provide a foundation for one of the most crucial results in elementary number theory: Bézout's Lemma.

Recall the definition of the greatest common divisor:

Definition 2.12. *Given two integers a and b, the greatest common divisor of a and b, denoted* $\gcd(a, b)$*, is the largest integer c such that $c \mid a$ and $c \mid b$.*

In Lean, the greatest common divisor of a and b can be written as `a.gcd b`. It is of natural number type.

Further, recall Bézout's Lemma:

Theorem 2.13. *Let a and b be integers. Then there exist integers x and y such that $ax + by = \gcd(a, b)$.*

In a typical mathematical paper, one might expect that the proof of Bézout's Lemma would be given at this point. However, due to the nature of this project, we refrain from proving Bézout's Lemma for now, as the formalisation in Lean requires several techniques not yet covered. Despite this, it is essential for the next formalisation, and so we will instead look to Mathlib for help.

A fundamental property of the greatest common divisor is that it characterises common divisors.

Theorem 2.14. *Let a and b be integers. An integer d divides both a and b if and only if d divides the greatest common divisor of a and b.*

Proof. We prove both directions of the biconditional separately. First assume $d \mid a$ and $d \mid b$. Then by Theorem 2.8, $d \mid ax + by$ for all x and y. But by Theorem 2.13, there must be some x and y such that $ax + by = \gcd(a, b)$. Hence $d \mid \gcd(a, b)$.

Now suppose $d \mid \gcd(a, b)$. Then there is some z such that $\gcd(a, b) = dz$. We know that $\gcd(a, b) \mid a$ and $\gcd(a, b) \mid b$ by definition of the greatest common divisor. But $\gcd(a, b) = dz$ so $dz \mid a$ and $dz \mid b$. Hence, by Lemma 2.5, $d \mid a$ and $d \mid b$. □

There is one significant obstacle that must be overcome to formalise any result involving elements of integer type when working with the greatest common divisor. Recall that the greatest common divisor has natural number type. Lean does not operate smoothly when working with both integers and natural numbers, or in fact, when working with any sets of variables that are not all of the equivalent type. Luckily it is possible to convert variables between types, using coercions.

In our problem statement in Lean, we write the following:

```
theorem dvd_gcd : d ∣ a ∧ d ∣ b ↔ d ∣ (a.gcd b : ℤ) := by
```

Observe how the greatest common divisor has been handled. Despite its given type being a natural number, it is forced to be treated as an integer.

This method of coercion is more robust than the more convenient method, which is to simply type '↑`a.gcd b`'. This approach attempts to assume the wanted type without user input using the context of the other variables, although it can be wrong, and so we opt not to use it — it is important to note however, as the Lean Infoview uses this style.

We proceed with the formalisation. Recall that we can adopt the `constructor` tactic to split the biconditional into two subgoals. The first subgoal is to prove that if d divides a and d divides b, then d divides $\gcd(a, b)$. This is an implication, with the antecedent containing a conjunction, thus we might opt to use `intro` followed by `rcases`. Nevertheless it is more practical to utilise the `rintro` tactic, which works exactly the same as the `rcases` tactic, but executes the duty of the `intro` tactic first.

```
constructor
· rintro ⟨h₁, h₂⟩
```

From here, it is easiest manipulate the goal directly. We use Theorem 2.13, which is given in Mathlib as `Int.gcd_eq_gcd_ab`, to rewrite the greatest common divisor as a linear combination of a and b, and then conclude that this is divisible by d by Theorem 2.8.

```
  rw[Int.gcd_eq_gcd_ab]
  exact my.dvd_add_of_two_mul h₁ h₂
```

For the other direction, notice that again `rintro` can be harnessed, with the `rcases` step of `rintro` being to expand a divisibility statement rather than split a conjunction. From here, a crucial tactic is exploited: the `have` tactic. This tactic allows us to create hypotheses as we please, provided of course that they are true in the context. The tactic works in a similar way to our approach to proving any result: we (optionally) provide an identifier (if we do not, the result is labelled as '`this`'), write the proposition, and then prove this proposition. Once it is proven, the hypothesis is added to our library of hypotheses to aid us in the main formalisation. This tactic unlocks a great deal of flexibility when working on formalising a proposition. We know that $\gcd(a, b)$ divides a and $\gcd(a, b)$ divides b, but these facts are not available to us until we explicitly define and prove them. We supply identifiers $\mathtt{h_2}$ and $\mathtt{h_3}$ respectively, and use results available to us (in Mathlib) to prove them.

```
· rintro ⟨z, h₁⟩
  have h₂ : (a.gcd b : ℤ) ∣ a := Int.gcd_dvd_left
  have h₃ : (a.gcd b : ℤ) ∣ b := Int.gcd_dvd_right
```

Notice that we did not include ':= by' in the `have` statements. Typically it would be included, however for `exact` statements it can be ignored.

To close the subgoal and thus conclude the formalisation it is a simple case of using `rw` and `exact`, as we have seen many times.

```
    rw[h₁] at h₂ h₃
    exact ⟨my.prod_left_dvd h₂, my.prod_left_dvd h₃⟩
```

Theorem 2.14 unlocks numerous results involving the greatest common divisor, not least a rather simple, yet vital property.

Lemma 2.15. *Let a and b be integers. Then* $\gcd(a, b) = \gcd(b, a)$.

Proof. Clearly $\gcd(a, b) \mid b$, $\gcd(a, b) \mid a$, $\gcd(b, a) \mid a$ and $\gcd(b, a) \mid b$. Therefore by Theorem 2.14 we can deduce that $\gcd(a, b) \mid \gcd(b, a)$ and $\gcd(b, a) \mid \gcd(a, b)$. Hence, as the greatest common divisor is non-negative, $\gcd(a, b) = \gcd(b, a)$ (the result that if a and b are natural numbers, and they divide each other, then $a = b$ is formalised in the Appendix). □

First observe that since the proposition only involves greatest common divisors, there is no need for coercion in the problem statement. However, we will be forced to coerce the greatest common divisor to integer type if we want to use Theorem 2.14, since its definition relies on integer types.

Our formalisation is crafted to rely heavily on the `have` tactic — this is not compulsory to conquer this formalisation, but for pedagogical reasons we will persevere with it. We could elect to define separate hypotheses for each of the four initial assertions in the proof, but for brevity it is preferable to directly proceed with deducing that $\gcd(a, b)$ divides $\gcd(b, a)$ and vice versa. The case $\gcd(a, b)$ divides $\gcd(b, a)$ will be discussed, its opposite follows in a similar way. The `have` tactic is written as usual, and Theorem 2.14 is called. Notice the '`.mp`' affixed to `my.dvd_gcd`. Its purpose is to specify the desired direction of the 'if and only if'. In this case we seek to use the forward direction — if we wish to do the opposite, '`.mpr`' must be used instead. It remains to supply to the theorem the hypotheses it requires: that $\gcd(a, b)$ divides b and a. We know that these results are given by `Int.gcd_dvd_right` and `Int.gcd_dvd_left` respectively.

```
lemma gcd_comm : a.gcd b = b.gcd a := by
  have h₁ : (a.gcd b : ℤ) ∣ (b.gcd a : ℤ) := my.dvd_gcd.mp ⟨
    Int.gcd_dvd_right, Int.gcd_dvd_left⟩
```

```
have h₂ : (b.gcd a : ℤ) ∣ (a.gcd b : ℤ) := my.dvd_gcd.mp ⟨
  Int.gcd_dvd_right, Int.gcd_dvd_left⟩
```

At this point it is clear that $\uparrow \gcd(a, b) = \uparrow \gcd(b, a)$. However, we do not desire the coercions; the goal does not cast its elements to integer types. Thus, we employ the `norm_cast` tactic, which removes any coercions, provided the expression contains elements of just one type. Finally we make use of a result given in the Appendix, that if a and b are natural numbers, then a divides b and b divides a if and only if a equals b.

```
norm_cast at *
exact my.dvd_iff_eq.mp ⟨h₁, h₂⟩
```

The next result has a similar formalisation.

Lemma 2.16. *Let a, b and m be integers. Then* $\gcd(a, b) = \gcd(a, am + b)$.

Proof. We have $\gcd(a, b) \mid a$ so $\gcd(a, b) \mid am$. Furthermore, $\gcd(a, b) \mid b$. Thus $\gcd(a, b) \mid am + b$ and so by Theorem 2.14, $\gcd(a, b) \mid \gcd(a, am + b)$.

$\gcd(a, am+b) \mid a$ so $\gcd(a, am+b) \mid am$. Also $\gcd(a, am+b) \mid am+b$, hence $\gcd(a, am+b) \mid b$. Therefore by Theorem 2.14, $\gcd(a, am+b) \mid \gcd(a, b)$ and so the two expressions divide each other and we can conclude $\gcd(a, am+b) = \gcd(a, b)$. □

The structure of this formalisation is exactly the same as seen in Lemma 2.15, just with slightly more complex expressions. We will briefly examine hypotheses $\mathtt{h_1}$ and $\mathtt{h_3}$. For the first, we want to prove that $\gcd(a, b)$ divides $am+b$, which we achieve by applying Theorem 2.9. Recall that this result requires two hypotheses: that $\gcd(a, b)$ divides am and that $\gcd(a, b)$ divides b. Divisibility of am is demonstrated by employing Lemma 2.4 which demands its own hypothesis: that $\gcd(a, b)$ divides a (which is true by `Int.gcd_dvd_left`). Divisibility of b is obvious by `Int.gcd_dvd_right`.

Hypothesis $\mathtt{h_3}$ is that $\gcd(a, am + b)$ divides b. This uses the result '`dvd_add_iff_dvd_both`' from the Appendix, which in the context of this proof, is that if $\gcd(a, am+b)$ divides $am+b$, then $\gcd(a, am+b)$ divides am if and only if $\gcd(a, am + b)$ divides b. We are only interested in the 'if' hence the `.mp`. We have $\gcd(a, am + b)$ divides $am + b$ by `Int.gcd_dvd_right`, and so it remains to prove that $\gcd(a, am + b)$ divides am. This is clear by Lemma 2.4 together with `Int.gcd_dvd_left`.

```
theorem gcd_eq_gcd_mul_add : a.gcd b = a.gcd (a * m + b) := by
  have h₁ : (a.gcd b : ℤ) ∣ (a * m + b) := my.dvd_add
    (my.dvd_mul_right_general Int.gcd_dvd_left) Int.gcd_dvd_right
  have h₂ : (a.gcd b : ℤ) ∣ (a.gcd (a * m + b) : ℤ) :=
    my.dvd_gcd.mp ⟨Int.gcd_dvd_left, h₁⟩
  have h₃ : (a.gcd (a * m + b) : ℤ) ∣ b := (my.dvd_add_iff_dvd_both
    Int.gcd_dvd_right).mp (my.dvd_mul_right_general Int.gcd_dvd_left)
  have h₄ : (a.gcd (a * m + b) : ℤ) ∣ (a.gcd b : ℤ) :=
    my.dvd_gcd.mp ⟨Int.gcd_dvd_left, h₃⟩
  norm_cast at h₂ h₄
  exact my.dvd_iff_eq.mp ⟨h₂, h₄⟩
```

Another important property of the greatest common divisor is its response to multiplication.

Lemma 2.17. *Let a and b be integers and x be an integer greater than zero. Then* $\gcd(xa, xb) = x\gcd(a, b)$.

Proof. Suppose that $\gcd(a, b) = 0$. Then $a = 0$ and $b = 0$ so $\gcd(xa, xb) = \gcd(0, 0) = 0$ and $x\gcd(a, b) = x\gcd(0, 0) = 0$.

Now suppose $\gcd(a, b) \neq 0$. By Theorem 2.14 we have that $x \mid \gcd(xa, xb)$, thus $\gcd(xa, xb) = xk$ for some $k \in \mathbb{Z}$. Since $xk \mid xa$ and $xk \mid xb$ we have $k \mid a$ and $k \mid b$, therefore $k \mid \gcd(a, b)$ by Theorem 2.14. On the other hand $\gcd(a, b) \mid a$ and $\gcd(a, b) \mid b$ so $x\gcd(a, b) \mid xa$ and $x\gcd(a, b) \mid xb$ thus by Theorem 2.14, $x\gcd(a, b) \mid \gcd(xa, xb) = xk$. Hence $\gcd(a, b) \mid k$.

We can conclude from above that $k = \gcd(a, b)$ or $k = -\gcd(a, b)$. If $k = \gcd(a, b)$ then $\gcd(xa, xb) = xk = x\gcd(a, b)$. If $k = -\gcd(a, b)$ then $\gcd(xa, xb) = xk = -x\gcd(a, b)$. But $\gcd(a, b) \geq 0$ so $xk \geq 0$ therefore $k \geq 0$, implying that $-\gcd(a, b) \geq 0$ i.e. $\gcd(a, b) \leq 0$ which is false, since the greatest common divisor is always nonnegative, and it cannot be zero by assumption. Hence $\gcd(xa, xb) = x\gcd(a, b)$. □

Notice that the proof involves checking two separate cases: $\gcd(a, b) = 0$ and $\gcd(a, b) \neq 0$. These two cases cover every possible value of the greatest common divisor, and thus they have a dichotomous relationship, allowing us to utilise the `by_cases` tactic. This tactic creates two subgoals: the first being to prove the proposition in question using the given case, and the second being to prove it using the negation. Proving both subgoals is sufficient to conclude that the proposition is true.

```
lemma gcd_mul (h₁ : x > 0) : ((x * a).gcd (x * b) : ℤ) = (x * a.gcd
    b : ℤ) := by
  by_cases case : (a.gcd b : ℤ) = 0
```

As demonstrated in the written proof, the first subgoal is rather trivial. It is clear by the definition of the greatest common divisor that if $\gcd(a, b) = 0$ then $a = 0$ and $b = 0$. This fact is given by the Mathlib result '`gcd_eq_zero_iff.mp`' which requires explicit variables a and b. We use another new tactic, `obtain`, to generate a new hypothesis defining a and b to be zero — in this context, this tactic operates in an identical manner to the `have` tactic, but removes the need to explicitly state the hypothesis being proven. It is able to deduce the hypothesis by evaluating the result it is provided with.

```
  · obtain h₂ := (gcd_eq_zero_iff a b).mp case
```

Now we simply substitute a and b into the goal and simplify. Notice that, given a conjunction, we can access the left and right assumptions using '`.left`' and '`.right`'.

```
    rw[h₂.left, h₂.right]
    simp
```

That concludes the first subgoal. The second requires far more work. We begin by establishing that x divides the integer $\gcd(xa, xb)$. Note that this is contingent on `have`, as `obtain` would lack sufficient context to understand what we intend to define. We leverage `rcases` to rewrite this as an equality with an arbitrary integer k, and use `have` to acquire that xk divides xa and xb.

```
  · have h₂ : x ∣ ((x * a).gcd (x * b) : ℤ) := my.dvd_gcd.mp ⟨
    my.dvd_mul_right, my.dvd_mul_right⟩
    rcases h₂ with ⟨k, h₂⟩
    have h₃ : x * k ∣ x * a := by
      rw[← h₂]
      exact Int.gcd_dvd_left
    have h₄ : x * k ∣ x * b := by
      rw[← h₂]
      exact Int.gcd_dvd_right
```

Both $\mathtt{h_3}$ and $\mathtt{h_4}$ can be reduced to k divides a and b respectively, by applying a result from the Appendix, recognising that since $x > 0$, it must be that $x \neq 0$ (given by Mathlib result '`Int.ne_of_lt`').

```
apply my.mul_dvd_mul' (Int.ne_of_lt h₁).symm at h₃
apply my.mul_dvd_mul' (Int.ne_of_lt h₁).symm at h₄
```

One might notice the use of '`.symm`' following the aforementioned result. The result is exactly: 'if $a < b$ then $a \neq b$'. Discern that $\mathtt{h_1}$ is exactly $x > 0$; we require $0 < x$ to use the result normally. Implementing `.symm` reverses the direction of the components of the statement to 'if $b > a$ then $b \neq a$' which allows us to apply it to $\mathtt{h_1}$. As an additional remark, there is a generic `symm` tactic that reverses the specified hypothesis (or the goal if no specification is given) — we will see this later.

It is clear now that k divides $\gcd(a, b)$. In this case there is enough information available for `obtain` to be used.

```
obtain h₅ := my.dvd_gcd.mp ⟨h₃, h₄⟩
```

It remains to prove that $\gcd(a, b)$ divides k. It is quick to derive that $x \gcd(a, b)$ divides $\gcd(xa, xb)$ via a string of `have` usages. Thus $x \gcd(a, b)$ divides xk, and so $\gcd(a, b)$ divides k.

```
have h₆ : x * (a.gcd b : ℤ) ∣ x * a := my.mul_dvd_mul
(my.dvd_reflex) Int.gcd_dvd_left
have h₇ : x * (a.gcd b : ℤ) ∣ x * b := my.mul_dvd_mul
(my.dvd_reflex) Int.gcd_dvd_right
have h₈ : x * (a.gcd b : ℤ) ∣ ((x * a).gcd (x * b) : ℤ) :=
my.dvd_gcd.mp ⟨h₆, h₇⟩
rw [h₂] at h₈
apply my.mul_dvd_mul' (Int.ne_of_lt h₁).symm at h₈
```

By a theorem in the Appendix, '`dvd_iff_eq_or_neg_eq`', since k divides $\gcd(a, b)$ and vice versa, and we are working in the integers, it must be that $k = \gcd(a, b)$ or $k = -\gcd(a, b)$. This is again a simple use of `obtain`. It is left to verify both cases. This requires splitting $\mathtt{h_9}$ into two separate subgoals, with the respective cases given as an assumption in their subgoal, with the goal inside the subgoal simply being the overall goal — this can be achieved by utilising `obtain` in a slightly different fashion.

```
obtain h₉ := my.dvd_iff_eq_or_neg_eq.mp ⟨h₅, h₈⟩
obtain h₁₀ ∣ h₁₁ := h₉
```

The first subgoal has $\mathtt{h_{10}}$, that $k = \gcd(a, b)$. This subgoal is trivial.

We aim to prove that $k = -\gcd(a, b)$ cannot happen. Thus we lead with `exfalso`, which changes the goal to 'False' — we aim to find two properties that directly contradict each other.

```
    · rw [h₂, h₁₀]
    · exfalso
```

Finding two such properties is straightforward but requires some meticulous code. Firstly we can declare that $\gcd(xa, xb) \geq 0$ by a Mathlib result. Thus $xk \geq 0$ using a previous assumption, and so $k \geq 0$ (using Mathlib result '`nonneg_of_mul_nonneg_right`' together with $x > 0$). Now since $k \geq 0$ and $k = -\gcd(a, b)$, using `rw` we obtain $-\gcd(a, b) \geq 0$. Using '`Int.nonpos_of_neg_nonneg`' we can rewrite this as $\gcd(a, b) \leq 0$. However, by '`Int.ofNat_zero_le`', $\gcd(a, b) \geq 0$, so combining the two facts we have $\gcd(a, b) = 0$. This directly contradicts that the greatest common divisor of a and b is nonzero, and so we can apply `exact` together with the contradicting hypotheses to close the goal, as the contradiction implies 'False'.

```
      have h₁₂ : ((x * a).gcd (x * b) : ℤ) ≥ 0 := by
        norm_cast
        exact Nat.zero_le ((x * a).gcd (x * b))
      rw[h₂] at h₁₂
      obtain h₁₃ := nonneg_of_mul_nonneg_right h₁₂ h₁
      rw[h₁₁] at h₁₃
      apply Int.nonpos_of_neg_nonneg at h₁₃
      obtain h₁₄ := Int.ofNat_zero_le (a.gcd b)
      obtain h₁₅ := Int.le_antisymm h₁₃ h₁₄
      exact case h₁₅
```

From Lemma 2.17 we can derive a useful result.

Corollary 2.18. *Let a and b be integers with at least one nonzero. If $\gcd(a, b) = d$ then $\gcd\left(\frac{a}{d}, \frac{b}{d}\right) = 1$.*

Proof. Clearly $\gcd(a, b) \mid a, b$ so $d \mid a, b$. Therefore there exists $k, l \in \mathbb{Z}$ such that $a = dk$ and $b = dl$. Substituting into our equation we have $\gcd(dk, dl) = d$, and so by Lemma 2.17, $d\gcd(k, l) = d$. Dividing by d, we get $\gcd(k, l) = 1$. Now $k = \frac{a}{d}$ and $l = \frac{b}{d}$, thus $\gcd(\frac{a}{d}, \frac{b}{d}) = 1$. □

The formalisation is generally quite simple, as the heavy lifting is carried out by the previous Lemma. Initially we lay some groundwork for the rest of the formalisation — specifically we acquire that $d \neq 0$ and $d > 0$. The code

is quite self-explanatory; note that 'Nat.zero_lt_of_ne_zero' is a Mathlib result stating that if $a \neq 0$ then $0 < a$ for a natural number a.

```
lemma gcd_div (h₁ : (a.gcd b : ℤ) ≠ 0) (h₂ : (a.gcd b : ℤ) = d) :
    ((a / d).gcd (b / d) : ℤ) = 1 := by
  have h₃ : (a.gcd b : ℤ) > 0 := by
    norm_cast at *
    exact Nat.zero_lt_of_ne_zero h₁
  rw[h₂] at h₃
  rw[h₂] at h₁
```

As we have now seen on numerous occasions, we generate hypotheses stating that $\gcd(a, b)$ divides a and b. This is then rewritten in terms of d, and another frequently encountered method is used — manipulation of a divisor into an equality using `rcases`.

```
  have h₄ : (a.gcd b : ℤ) ∣ a := Int.gcd_dvd_left
  have h₅ : (a.gcd b : ℤ) ∣ b := Int.gcd_dvd_right
  rw[h₂] at h₄ h₅
  rcases h₄ with ⟨k, h₄⟩
  rcases h₅ with ⟨l, h₅⟩
```

We then rewrite $\mathtt{h_2}$, first substituting in $a = dk$ and $b = dl$, and then applying Lemma 2.17.

```
  rw[h₄, h₅, my.gcd_mul h₃] at h₂
```

Utilising the Mathlib result 'Int.eq_one_of_mul_eq_self_right' (which states that for integers a and b, if b is nonzero, and if $ba = b$, then $a = 1$) paired with `apply` yields that the integer form of the greatest common divisor of k and l is 1.

```
  apply Int.eq_one_of_mul_eq_self_right h₁ at h₂
```

Finally, we rearrange $a = dk$ and $b = dl$ to make d the subject, again combining `apply` with a Mathlib result — 'Int.eq_ediv_of_mul_eq_right'. This result claims that for integers a, b and c, if a is nonzero and if $ab = c$, then $b = \frac{c}{a}$. Substituting this into $\gcd(k, l) = 1$, we reach our desired conclusion.

```
  symm at h₄ h₅
  apply Int.eq_ediv_of_mul_eq_right h₁ at h₄
  apply Int.eq_ediv_of_mul_eq_right h₁ at h₅
```

```
  rw[h₄, h₅] at h₂
  assumption
```

Note that we employ `symm` to facilitate a straight forward application of `Int.eq_ediv_of_mul_eq_right`.

2.4 Bézout's Lemma

Now that we have established a significant repertoire of tactics, techniques and results, we can begin to build towards the formalisation of Bézout's Lemma (Theorem 2.13). To be clear, we emphasise that our approach is based on [16].

To begin with, we require a pivotal result in modular arithmetic. A more in-depth analysis of this type of theory can be found in Chapter 5.

We define a new theorem, proposing that for integers a and b, if b is nonzero, then the (natural) absolute value of a mod b (defined $a\%b$) is less than the (natural) absolute value of b. To clarify, by a natural absolute value, we refer to the type of the absolute value — in Lean, $|a|$ is given integer type, but '`.natAbs`' forces $|a|$ to be of natural number type. Later it will become clear as to why we require this theorem.

It is left as an exercise for the reader to investigate the mechanics of this formalisation (per Chapter 1, Loogle is recommended when one is aware of the name of a Mathlib result). We choose not to discuss this specific theorem in much detail as it is more a statement of convenience; our formalisation of this fact relies heavily on Mathlib results that already do much of the heavy lifting.

```
theorem natAbs_mod_lt (h₁ : b ≠ 0) : (a % b).natAbs < b.natAbs := by
  rw [Int.natAbs_lt_iff_sq_lt, sq_lt_sq]
  rw [abs_of_nonneg]
  exact Int.emod_lt a h₁
  exact Int.emod_nonneg a h₁
```

To prove Bézout's Lemma, we will prove the existence of two integers satisfying the given condition. These integers can be calculated via the Extended Euclidean Algorithm. Recall the Euclidean Algorithm — an algorithm to

compute the greatest common divisor of two integers a and b:

$$\begin{aligned}
a &= bq_0 + r_0, \quad 0 \leq r_0 < |b|, \\
b &= r_0 q_1 + r_1, \quad 0 \leq r_1 < |r_0|, \\
r_0 &= r_1 q_2 + r_2, \quad 0 \leq r_2 < |r_1|, \\
&\vdots \\
r_{n-2} &= r_{n-1} q_n + r_n, \quad 0 \leq r_n < |r_{n-1}|, \\
r_{n-1} &= r_n q_{n+1} + 0.
\end{aligned}$$

We obtain $\gcd(a, b) = r_n$ for some integer $n \geq 0$. The Extended Euclidean Algorithm involves forward-substituting the following expressions to express the greatest common divisor as a linear combination of a and b:

$$\begin{aligned}
r_n &= r_{n-2} - r_{n-1} q_n, \\
r_{n-1} &= r_{n-3} - r_{n-2} q_{n-1}, \\
&\vdots \\
r_2 &= r_0 - r_1 q_2, \\
r_1 &= b - r_0 q_1, \\
r_0 &= a - bq_0.
\end{aligned}$$

Notice that from the Euclidean Algorithm we are able to deduce that:

$$\begin{aligned}
\gcd(a, b) &= \gcd(b, r_0) \\
&= \gcd(r_0, r_1) \\
&\vdots \\
&= \gcd(r_{n-1}, r_n) \\
&= \gcd(r_n, 0) \\
&= r_n
\end{aligned}$$

At each step of the Extended Euclidean Algorithm there is a linear combination of integers directly corresponding to the pairs of integers sharing greatest common divisors above, and equal to the greatest common divisor of a and b. Suppose now that any pair of these integers is defined as a and b. We can derive functions for calculating the integers associated with a and b such that the linear combination is equal to r_n. These functions with the initial values of a and b yield exactly the integers inserted into Bézout's Lemma.

Definition 2.19. *The functions calculating the integers associated with a and b such that the linear combination is equal to r_n are as follows:*

$$L(a,b) = \begin{cases} R(b, a \bmod b) & 0 < b \\ R(b, a \bmod -b) & b < 0 \\ 1 & b = 0 \text{ and } 0 \leq a \\ -1 & b = 0 \text{ and } a < 0. \end{cases}$$

$$R(a,b) = \begin{cases} L(b, a \bmod b) - \left\lfloor \frac{a}{b} \right\rfloor R(b, a \bmod b) & 0 < b \\ L(b, a \bmod -b) + \left\lfloor \frac{a}{b} \right\rfloor R(b, a \bmod -b) & b < 0 \\ 0 & b = 0. \end{cases}$$

Note that the functions are interdependent on each other. Typically in Lean it is impossible to utilise a definition or result formalised at a later stage. Hence, we require a special environment, since one function must be defined below the other — we work inside of a `mutual`...`end` pair.

Both functions are defined almost identically, thus we will discuss only $L(a,b)$. First we write the initial statement. We write `def` rather than `theorem` or `lemma` and provide explicit variables a and b (defined to have integer type). Instead of a typical proposition after the colon, we instead simply write '$\mathbb{Z}$' to characterise the function as having integer output.

```
def L (a b : ℤ) : ℤ :=
```

Up to this point, we have discovered that Lean differs significantly from a traditional programming language. Nevertheless, defining $L(a,b)$ involves some familiar aspects — in particular, `if`...`else` statements form the foundations of the definition. The code is self-explanatory — inspect each line defining $L(a,b)$ and translate it to code with a continuation of the `if`...`else`.

```
  if h₁ : b > 0 then
    R b (a % b)
  else if h₁: b < 0 then
    R b (a % (-b))
  else if a ≥ 0 then
    1
  else
    -1
```

The definition is not complete until it is demonstrated that the function is well-founded — that is, we must check that it does not loop infinitely. This

involves the 'termination_by' tactic, which allows us to specify a measure for a recursive function that demonstrates that it will terminate. The termination measure of choice for $L(a, b)$ is `b.natAbs`. It remains to validate that `(a % b).natAbs < b.natAbs` for when b is both greater than and less than zero. Since b is strictly less than or greater than zero, it is not equal to zero, and so by `natAbs_mod_lt`, we have our result.

```
def L (a b : ℤ) : ℤ :=
  if h₁ : b > 0 then
    have h₂ : b ≠ 0 := (Int.ne_of_lt h₁).symm
    have : (a % b).natAbs < b.natAbs := natAbs_mod_lt h₂
    R b (a % b)
  else if h₁: b < 0 then
    have h₂ : b ≠ 0 := (Int.ne_of_gt h₁).symm
    have : (a % b).natAbs < b.natAbs := natAbs_mod_lt h₂
    R b (a % (-b))
  else if a ≥ 0 then
    1
  else
    -1
termination_by b.natAbs
```

For reference, $R(a, b)$ is defined in Lean as follows:

```
def R (a b : ℤ) : ℤ :=
  if h₁ : b > 0 then
    have h₂ : b ≠ 0 := (Int.ne_of_lt h₁).symm
    have : (a % b).natAbs < b.natAbs := natAbs_mod_lt h₂
    L b (a % b) - (a / b) * R b (a % b)
  else if h₁ : b < 0 then
    have h₂ : b ≠ 0 := (Int.ne_of_gt h₁).symm
    have : (a % b).natAbs < b.natAbs := natAbs_mod_lt h₂
    L b (a % (-b)) + (a / (-b)) * R b (a % (-b))
  else
    0
termination_by b.natAbs
```

It should be noted that the forward slash `/` is indeed the floor function. Earlier it was used as integer division, which was true in the context of the problem, but in this case divisibility of a by b is not assumed.

Now we proceed with the final formalisation of this chapter.

Proof of Theorem 2.13. Let a and b be integers and $L(a,b)$ and $R(a,b)$ be the functions in Definition 2.19. We prove that $L(a,b)\cdot a + R(a,b)\cdot b = \gcd(a,b)$.

We verify that the equation holds for each of the four cases presented in the definition of $L(a,b)$. Proceeding by strong induction on b with inductive hypothesis $L(b,r)b + R(b,r)r = \gcd(b,r)$, first we examine the case where b is greater than 0. Let $q = \left\lfloor \frac{a}{b} \right\rfloor$ and $r = a \bmod b$. It is easy to see that $a = bq + r$. We have $L(a,b) = R(b,r)$ and $R(a,b) = L(b,r) - qR(b,r)$. Therefore

$$\begin{aligned} L(a,b)a + R(a,b)b &= R(b,r)a + (L(b,r) - qR(b,r))\,b \\ &= R(b,r)(bq + r) + (L(b,r) - qR(b,r))b \\ &= R(b,r)r + bL(b,r) \\ &= \gcd(b,r) \end{aligned}$$

Rearranging $a = bq + r$ we obtain $r = a - bq$, and substituting this into $\gcd(b,r)$ we arrive at $\gcd(b, a - bq)$, which simplifies to $\gcd(b,a)$ by Lemma 2.16. Hence, $L(a,b)a + R(a,b)b = \gcd(b,a) = \gcd(a,b)$ and so the proposition is true for $b > 0$.

The remaining verification is omitted. The case $b < 0$ is similar to above, and the other two are trivial. □

We begin the formalisation with a substitution of the actual meanings of $L(a,b)$ and $R(a,b)$ into the goal. This gives rise to a quite complex new goal containing multiple `if...else` statements, although this is quickly fixed via a new tactic, '`split_ifs`'. This tactic is somewhat self-explanatory — each part of the `if...else` statement in the goal is split into separate subgoals, with each subgoal containing the respective hypotheses describing the case in question and eliminating any other case already solved. For example, the second case of $L(a,b)$ is that $b < 0$. Note that at this point the case $b > 0$ will have been covered. Hence, the hypotheses for this subgoal are $b < 0$ and $\neg b > 0$. For ease of formalisation, we write '`<;> push_neg`' to reformulate any negations in the produced subgoals into the correct form, for example $\neg b > 0$ is rewritten as $b \leq 0$. To emphasise, the role of `<;>` is to run `push_neg` on each subgoal rather than just the first, and can be used with other tactics, not just `push_neg`. It is also specified to do this '`at *`' so `push_neg` is run on every hypothesis and every goal of every subgoal.

```
theorem bezouts_lemma (a b : ℤ) : L a b * a + R a b * b = (a.gcd b
    : ℤ) := by
  rw [R, L]
```

```
split_ifs with h₁ h₂ h₃
<;> push_neg at *
```

To clarify, `split_ifs` requires three identifiers: $\mathtt{h}_1$, $\mathtt{h}_2$ and $\mathtt{h}_3$, as the maximum number of hypotheses required for the subgoal to be rigorous is three.

Now we proceed with the first subgoal, that is, the proposition is true for $b < 0$. Note that the corresponding values of $L(a, b)$ and $R(a, b)$ are already substituted into the goal for us. We use '`set`', a tactic that allows us to define a new element based on existing elements, and exchanges the indicated element from the goal for its specified equal. In this case, we set `q` and `r` to their required values. Subsequently `simp` is employed to reduce the goal into a clearer form (this form being the form seen in the written proof).

```
· set q := a / b
  set r := a % b
  simp
```

Utilising `obtain` we fetch the inductive hypothesis, and to finish the general setup, use `have` to acquire $bq + r = a$, exploiting the convenient Mathlib result '`Int.ediv_add_emod`', which states that for integers a and b we have: `b * (a / b) + a % b = a`.

```
  obtain ih := bezouts_lemma b r
  have h₅ : b * q + r = a := Int.ediv_add_emod a b
```

From this foundational work the formalisation of this subgoal becomes a simple amalgamation of tactics and results that we are now experienced with.

```
  nth_rw 1 [← h₅]
  ring_nf
  rw[add_comm, mul_comm, ih]
  have h₆ : r = b * -q + a := by linarith
  rw[h₆]
  nth_rw 2 [my.gcd_comm]
  rw[← my.gcd_eq_gcd_mul_add]
```

The observant reader may notice that we take advantage of `my.gcd_eq_gcd_mul_add`, which requires `dvd_gcd`, which uses `Int.gcd_eq_gcd_ab` (i.e. Mathlib's Bézout's Lemma), thus creating a cyclic argument. Of course this

is not ideal, but for the purposes of this paper it is not particularly problematic — the results we elected to formalise were chosen based on their potential to enhance understanding of Lean and are obviously true. A formalisation of `dvd_gcd` not reliant on Bézout's Lemma is outside the scope of this project, hence why we do not provide one.

There is one final detail that must be addressed. The inductive hypothesis is given by `bezouts_lemma b r`. This makes our formalisation recursive, and thus it is required, as in Definition 2.19, to ensure that there is no infinite loop. Thus we write `termination_by` at the conclusion of the formalisation (i.e. after every subgoal has been proven) and pair it with `b.natAbs`. Then, at an appropriate position inside the subgoal, we prove that `r.natAbs < b.natAbs`, which is trivial by `natAbs_mod_lt`.

```
  have h₄ : b ≠ 0 := (Int.ne_of_lt h₁).symm
  have termination : r.natAbs < b.natAbs := natAbs_mod_lt h₄
```

The second subgoal is almost identical to the first subgoal, of course with slightly modified q and r.

```
· set q := a / (-b)
  set r := a % (-b)
  simp
  have h₄ : b ≠ 0 := Ne.symm (Int.ne_of_gt h₂)
  have termination : (a % b).natAbs < b.natAbs := natAbs_mod_lt h₄
  obtain ih := bezouts_lemma b r
  have h₅ : (-b) * q + r = a := Int.ediv_add_emod a (-b)
  nth_rw 1 [← h₅]
  ring_nf
  rw[add_comm, mul_comm, ih]
  have h₆ : r = b * q + a := by linarith
  rw[h₆]
  nth_rw 2 [my.gcd_comm]
  rw[← my.gcd_eq_gcd_mul_add]
```

The final two subgoals are trivial.

```
· ring_nf
  have : b = 0 := Int.le_antisymm h₁ h₂
  rw[this]
  norm_num
  rw[abs_of_nonneg h₃]
· ring_nf
```

```
    have : b = 0 := Int.le_antisymm h₁ h₂
    rw[this]
    norm_num
    rw[abs_of_neg h₃]
termination_by b.natAbs
```

Thus we have proven Bézout's Lemma, although to stay true to the exact wording of the theorem, we should formalise one last proposition:

```
theorem bezouts_lemma' (a b : ℤ) : ∃ x y : ℤ, x * a + y * b =
    (a.gcd b : ℤ) := by
  use L a b, R a b
  apply bezouts_lemma
```

Chapter 3

The Binomial Theorem

In this chapter we formalise the prerequisites of the Binomial Theorem in Lean, as well as the Binomial Theorem itself. Although the Binomial Theorem is not strictly number-theoretic, it has applications in number theory (including in proofs that we will attempt to formalise later), and serves as an excellent bridge for this report.

At this stage the reader should have gained some understanding of Lean and should be able to grasp the general idea of the formalisations presented. This in combination with the fact that a new tactic is introduced that significantly improves the clarity of the code results (in general) in less explanation of code being provided.

3.1 The Binomial Coefficient

Recall the definition of the binomial coefficient.

Definition 3.1. *Given natural numbers n and k, the binomial coefficient describes the number of ways of choosing k elements from n distinct elements. It is denoted by $\binom{n}{k}$, and is described by the formula:*

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}.$$

Throughout this chapter, n and k are given natural number type. We define the binomial coefficient in Lean as follows:

```
def choose (n k : ℕ) : ℤ := n.factorial / (k.factorial * (n -
    k).factorial)
```

This takes two natural numbers n and k as inputs, and returns an integer computed using the expression.

To accustom ourselves with this definition (and to formalise an equation used in a proof in Chapter 4, which in turn is used in a proof in Chapter 5), we shall prove the following.

Lemma 3.2. *Let n and k be natural numbers with $0 < k \leq n$. Then:*

$$k \cdot \binom{n}{k} = n \cdot \binom{n-1}{k-1}$$

Proof.

$$\begin{aligned} k \cdot \binom{n}{k} &= k \cdot \frac{n!}{k!(n-k)!} \\ &= \frac{kn!}{k!(n-k)!} \\ &= \frac{k(n(n-1)!)}{k(k-1)!(n-k)!} \\ &= \frac{n(n-1)!}{(k-1)!(n-k)!} \\ &= n \cdot \frac{(n-1)!}{(k-1)!(n-1-(k-1))!} \\ &= n \cdot \binom{n-1}{k-1} \end{aligned}$$

□

The linear style of this proof makes it an excellent choice for the `calc` tactic. The `calc` tactic allows us to go step by step through solving an equality, proving each step using our knowledge of Lean's tactics and our library of results, as usual. This tactic is especially powerful from the perspective of readability of code, thus enhancing efficiency producing and debugging code, as well as allowing for a more pleasant experience analysing the Lean code for the reader.

Some preparation is required before proceeding with `calc`. To start, we employ the `unfold` tactic, which substitutes the expression of the prescribed definition into the goal. In this case, we write `unfold choose` to replace occurrences of `choose n k` with its expression (as given in Definition 3.1). Following this, we must write `norm_cast`. This is due to the definition of

the binomial coefficient in Lean bestowing its output with integer type, thus each element of the expression is cast to integer type, yet we require natural numbers in this proposition.

```
lemma choose_helper (h₁ : k > 0) (h₂ : k ≤ n) : k * choose n k = n *
    choose (n - 1) (k - 1) := by
  unfold choose
  norm_cast
```

Now we initialise `calc`. We begin the first step of proving the equality by writing '`_=`'. The underscore typically is representative of the right-hand side of the equality in the previous step, however since we do not have a previous step, the underscore represents the left-hand side of the goal.

After the equals sign we write the expression we would like to reach by the end of the step. Returning to the proof of Lemma 3.2, we see that the first step is to simply (although not so simply in Lean) multiply the fraction by k to obtain k in the numerator. Following this we write '`:= by`' as is conventional in any approach immediately preceding a proof, and then simply formulate our proof as usual.

```
  calc
    _ = k * n.factorial / (k.factorial * (n - k).factorial) := by
```

To verify that the equality is indeed accurate, we use the Mathlib result '`Nat.mul_div_assoc`' which states that for natural numbers a, b and c, if a divides c then $(bc)/a = b(c/a)$. After performing the rewrite, it remains to prove that $k!(n-k)!$ divides $n!$, which is solved by '`Nat.factorial_mul_factorial_dvd_factorial`' provided k is less than or equal to n.

```
      rw[← Nat.mul_div_assoc]
      exact Nat.factorial_mul_factorial_dvd_factorial h₂
```

Having demonstrated the correctness of the equality, we can move onto the next step. On account of the enhanced readability of the formalisation as well as its overall simplicity, we leave it is an exercise to study the remainder of the code. Again refer to Loogle for information on any new Mathlib results in use.

```
    _ = k * (n * (n - 1).factorial) / (k * (k - 1).factorial * (n -
  k).factorial) := by
      rw[Nat.mul_factorial_pred (Nat.lt_of_lt_of_le h₁ h₂),
  Nat.mul_factorial_pred h₁]
```

```
  _ = n * (n - 1).factorial / ((k - 1).factorial * (n -
k).factorial) := by
    rw[mul_assoc, Nat.mul_div_mul_left]
    assumption
  _ = n * ((n - 1).factorial / ((k - 1).factorial * (n - 1 - (k -
1)).factorial)) := by
    rw[Nat.mul_div_assoc, Nat.sub_sub_sub_cancel_right h1]
    rw[← Nat.sub_sub_sub_cancel_right h1, Nat.succ_eq_add_one,
zero_add]
    have h3 : (k - 1).factorial * (n - 1 - (k - 1)).factorial | (n
- 1).factorial := by
      have h4 : k - 1 ≤ n - 1 := by
        exact Nat.sub_le_sub_right h2 1
      exact Nat.factorial_mul_factorial_dvd_factorial h4
    exact h3
```

3.2 Pascal's Rule

Having become familiar with both our Lean definition of the binomial coefficient and the `calc` tactic, the final measure before attempting to prove the binomial theorem is to prove a contingent result: Pascal's Rule.

Theorem 3.3. *Let n and k be natural numbers with $0 < k \leq n$. Then:*

$$\binom{n}{k} + \binom{n}{k-1} = \binom{n+1}{k}.$$

Proof.

$$
\begin{aligned}
\binom{n}{k} + \binom{n}{k-1} &= \frac{n!}{k!(n-k)!} + \frac{n!}{(k-1)!(n-(k-1))!} \\
&= \frac{n!(n-k+1)}{k!(n-k)!(n-k+1)} + \frac{n!}{(k-1)!(n-k+1)!} \\
&= \frac{n!(n-k+1)}{k!(n-k)!(n-k+1)} + \frac{n!k}{(k-1)!(n-k+1)!k} \\
&= \frac{n!(n-k+1)}{k!(n-k+1)!} + \frac{n!k}{(n-k+1)!k} \\
&= \frac{n!(n-k+1)}{k!(n-k+1)!} + \frac{n!k}{k!(n-k+1)!} \\
&= \frac{n!(n-k+1) + n!k}{k!(n-k+1)!} \\
&= \frac{n!(n+1)}{k!(n-k+1)!} \\
&= \frac{(n+1)!}{k!(n+1-k)!} \\
&= \binom{n+1}{k}
\end{aligned}
$$

□

Notice the linearity of the proof — again we can utilise `calc`. In fact, the formalisation is rather similar to the formalisation of Lemma 3.2, opening with the exact same code.

```
theorem pascals_rule (h₁ : k > 0) (h₂ : k ≤ n) : choose n k +
    choose n (k - 1) = choose (n + 1) k := by
  unfold choose
  norm_cast
  calc
```

Once more we can leave it to the reader to study this formalisation due to its user-friendliness.

```
    _ = (n.factorial * (n - k + 1)) / ((k.factorial * (n -
   k).factorial) * (n - k + 1)) + n.factorial / ((k - 1).factorial *
    (n - (k - 1)).factorial) := by
      rw[Nat.mul_div_mul_right]
      exact Nat.zero_lt_succ (n - k)
```

```
  _ = n.factorial * (n - k + 1) / (k.factorial * (n -
k).factorial * (n - k + 1)) +
    n.factorial * k / ((k - 1).factorial * (n - k + 1).factorial *
 k) := by
    rw[tsub_tsub_assoc h₂ h₁, Nat.succ_eq_add_one, zero_add,
Nat.mul_div_mul_right n.factorial ((k - 1).factorial * (n - k +
1).factorial) h₁]
  _ = n.factorial * (n - k + 1) / (k.factorial * (n - k +
1).factorial) +
    n.factorial * k / ((k - 1).factorial * (n - k + 1).factorial *
 k) := by
    nth_rw 1 [mul_assoc]
    nth_rw 3 [mul_comm]
    nth_rw 1 [← Nat.factorial_succ (n - k)]
  _ = n.factorial * (n - k + 1) / (k.factorial * (n - k +
1).factorial) +
    n.factorial * k / (k.factorial * (n - k + 1).factorial) := by
    nth_rw 4 [mul_comm]
    nth_rw 1 [← mul_assoc]
    have : k = k - 1 + 1 := by
      exact (Nat.sub_eq_iff_eq_add h₁).mp rfl
    nth_rw 5 [this]
    rw[← Nat.factorial_succ, ← this]
  _ = (n.factorial * (n - k + 1) + n.factorial * k) /
(k.factorial * (n - k + 1).factorial) := by
    rw[← Nat.add_div_of_dvd_right]
    rw[Nat.factorial_succ (n - k)]
    nth_rw 2 [mul_comm]
    rw[← mul_assoc]
    have h₃ : k.factorial * (n - k).factorial ∣ n.factorial := by
      apply Nat.factorial_mul_factorial_dvd_factorial h₂
    have h₄ : n - k + 1 ∣ n - k + 1 := my.dvd_reflex
    exact my.mul_dvd_mul h₃ h₄
  _ = n.factorial * (n + 1) / (k.factorial * (n - k +
1).factorial) := by
    rw[← Nat.mul_add n.factorial, ← Nat.sub_add_comm h₂,
Nat.sub_add_cancel (Nat.le_add_right_of_le h₂)]
  _ = (n + 1).factorial / (k.factorial * (n + 1 - k).factorial) :=
 by
    rw[← Nat.sub_add_comm h₂]
    rw[mul_comm]
    rw[Nat.factorial_succ]
```

3.3 The Binomial Theorem

We conclude this chapter with a formalisation of the binomial theorem. This will be crucial for the final formalisation in this paper. We define a and b to be of integer type.

Theorem 3.4. *Let a and b be integers and let n be a natural number. Then:*

$$(a+b)^n = \sum_{i=0}^{n} a^{(n-i)} b^i \binom{n}{i}.$$

Proof. We proceed by induction. The base case is trivial: if $n = 0$, then $(a+b)^0 = 1$ and

$$\sum_{i=0}^{0} a^{(0-i)} b^i \binom{0}{i} = a^0 b^0 \binom{0}{0} = 1.$$

Assume the proposition is true for $n = k$. That is,

$$(a+b)^k = \sum_{i=0}^{k} a^{(k-i)} b^i \binom{k}{i}.$$

Then for $n = k+1$:

$$\begin{aligned}
(a+b)^{k+1} &= (a+b)(a+b)^k \\
&= (a+b) \sum_{i=0}^{k} a^{(k-i)} b^i \binom{k}{i} \\
&= \sum_{i=0}^{k} a \left(a^{(k-i)} b^i \binom{k}{i} \right) + \sum_{i=0}^{k} b \left(a^{(k-i)} b^i \binom{k}{i} \right) \\
&= \sum_{i=0}^{k} a^{(k+1-i)} b^i \binom{k}{i} + \sum_{i=0}^{k} b \left(a^{(k-i)} b^i \binom{k}{i} \right) \\
&= \sum_{i=0}^{k} a^{(k+1-i)} b^i \binom{k}{i} + \sum_{i=0}^{k} a^{(k-i)} b^{i+1} \binom{k}{i} \\
&= a^{k+1} + \sum_{i=1}^{k} a^{(k+1-i)} b^i \binom{k}{i} + \sum_{i=0}^{k} a^{(k-i)} b^{i+1} \binom{k}{i} \\
&= \sum_{i=0}^{k} a^{(k+1-i)} b^i \binom{k}{i} + \sum_{i=0}^{k} a^{(k-i)} b^{i+1} \binom{k}{i}
\end{aligned}$$

$$
\begin{aligned}
&= a^{k+1} + \sum_{i=1}^{k} a^{(k+1-i)} b^i \binom{k}{i} + \left(\sum_{i=0}^{k-1} a^{(k-i)} b^{i+1} \binom{k}{i} \right) + b^{k+1} \\
&= a^{k+1} + \sum_{i=1}^{k} a^{(k+1-i)} b^i \binom{k}{i} + \left(\sum_{j=1}^{k} a^{(k+1-j)} b^j \binom{k}{j-1} \right) + b^{k+1} \\
&= a^{k+1} + \left(\sum_{i=1}^{k} a^{(k+1-i)} b^i \left(\binom{k}{i} + \binom{k}{i-1} \right) \right) + b^{k+1} \\
&= a^{k+1} + \left(\sum_{i=1}^{k} a^{(k+1-i)} b^i \binom{k+1}{i} \right) + b^{k+1} \quad \text{[By Theorem 3.3]} \\
&= \left(\sum_{i=0}^{k} a^{(k+1-i)} b^i \binom{k+1}{i} \right) + b^{k+1} \\
&= \sum_{i=0}^{k+1} a^{(k+1-i)} b^i \binom{k+1}{i}.
\end{aligned}
$$

□

The formalisation is among the most prolonged in this paper. It is clear that our starting point must be the `induction'` tactic. The base case is simple, unfolding `choose` and writing `simp` is enough to resolve it.

```
theorem binomial_theorem : (a + b) ^ n = Σ i ∈ Finset.range (n +
    1), a ^ (n - i) * b ^ (i) * choose n i := by
  induction' n with k ih
  · unfold choose
    simp
```

The inductive step can be solved using `calc`. Since it is the final result of the chapter, and quite substantial, we supply some interpretation of this section of the code.

The first three steps are trivial, harnessing the `rw` tactic with simple Mathlib results, including `pow_succ` and `Finset.mul_sum`, which prove that $a^{k+1} = a \cdot a^k$ and (for some function f) $a \sum_i f(i) = \sum_i af(i)$, respectively.

```
  · calc
      _ = (a + b) * (a + b) ^ k := by rw[pow_succ, mul_comm]
      _ = (a + b) * Σ i ∈ Finset.range (k + 1), a ^ (k - i) * b ^ i
    * choose k i := by rw[ih]
```

```
    _ = Σ i ∈ Finset.range (k + 1), a * (a ^ (k - i) * b ^ i *
choose k i) +
      Σ i ∈ Finset.range (k + 1), b * (a ^ (k - i) * b ^ i *
choose k i) := by
    rw [add_mul, Finset.mul_sum, Finset.mul_sum]
```

Now it is necessary to manipulate an expression inside of a summation. This is not directly possible in Lean, so we must remove the summation entirely first. This can be achieved through `Finset.sum_congr`, which states that for two finite sets S_1 and S_2, if $S_1 = S_2$ then for all i in S_2, $f(i) = g(i)$ implies $\sum_{i \in S_1} f(i) = \sum_{i \in S_2} g(i)$. If we let $S_2 = S_1$, then this result reduces to proving that the expression in the summation can be freely manipulated whilst maintaining mathematical accuracy — this explains the '`rfl`' paired with the result. It refers to reflexivity, i.e. that $S_1 = S_1$ (in this context, S_1 alludes to the first k natural numbers).

```
    _ = Σ i ∈ Finset.range (k + 1), a ^ (k + 1 - i) * b ^ i *
choose k i +
      Σ i ∈ Finset.range (k + 1), b * (a ^ (k - i) * b ^ i *
choose k i) := by
    rw[Finset.sum_congr rfl]
```

Additional work is required to finish this step. It remains to prove that for all i in the first k natural numbers, the summation can be manipulated as we demonstrate in the written proof. This is not too complicated, demanding the introduction of an arbitrary element i of the first k natural numbers, and some basic algebraic manipulation. Note that `Finset.mem_range_succ_iff` proves that since i is in `Finset.range (k + 1)`, we have that i is less than or equal to k.

```
    intro i h₁
    nth_rw 1 [← pow_one a]
    rw[← mul_assoc, ← mul_assoc, ← pow_add]
    nth_rw 1 [add_comm]
    rw[← Nat.sub_add_comm (Finset.mem_range_succ_iff.mp h₁)]
```

The next step follows very similarly.

```
    _ = Σ i ∈ Finset.range (k + 1), a ^ (k + 1 - i) * b ^ i *
choose k i +
      Σ i ∈ Finset.range (k + 1), a ^ (k - i) * b ^ (i + 1) *
choose k i := by
```

```
      nth_rw 2 [Finset.sum_congr rfl]
      intro i h₁
      nth_rw 1 [← pow_one b]
      rw[← mul_assoc, ← mul_assoc, mul_comm (b ^ 1) (a ^ (k -
i))]
      nth_rw 2 [mul_assoc]
      rw[← pow_add]
      nth_rw 1 [add_comm]
```

Next we aim to remove the case $i = 0$ from the left summation, which is unfortunately not as straightforward as one might hope. First of all, notice that `Finset.range` always generates a set beginning at zero. This is unacceptable, thus we swap to `Finset.Ico`, which allows for two inputs, a lower and upper bound, where the lower bound is defined by a 'greater than or equal to' condition, and the upper bound is a 'strictly less than' condition. One can perform this swap using the Mathlib result '`Finset.range_eq_Ico`'. Following this, we aim to use `Finset.sum_insert`, which for a set S and integer a is exactly:

$$a \notin S \implies \sum_{i \in \texttt{insert}\ a\ S} f(i) = f(a) + \sum_{i \in S} f(i).$$

Define `insert` a S to be the set $S \cap \{a\}$. Let S be the set of natural numbers from 1 to k (defined in Lean as '`Finset.Ico 1 (k + 1)`') and set a to be zero. Firstly it is clear that a is an element of `Finset.Ico 0 (k + 1)` (using `Finset.insert_eq_self.mp` — a inserted into `Finset.Ico 0 (k + 1)` does not change the set, thus a is in the set). Furthermore, a is not in `Finset.Ico 1 (k + 1)` — if we assume that a is in this set, then a would be greater than or equal to one and less than $k + 1$ (by `Finset.mem_Ico`), but then zero would be greater than or equal to one which is false. Now using these two facts together with `Finset.erase_eq_iff_eq_insert` it is possible to conclude that `Finset.Ico 0 (k + 1) = insert 0 (Finset.Ico 1 (k + 1))`. Rewriting the goal with this statement in mind, we can then apply `Finset.sum_insert` to remove the $i = 0$ case from the summation. Finally we simplify.

```
    _ = a ^ (k + 1) + Σ i ∈ Finset.Ico 1 (k + 1), a ^ (k + 1 - i)
* b ^ i * choose k i +
        Σ i ∈ Finset.range (k + 1), a ^ (k - i) * b ^ (i + 1) *
choose k i := by
      nth_rw 1 [Finset.range_eq_Ico]
      have h₁ : 0 ∈ Finset.Ico 0 (k + 1) :=
Finset.insert_eq_self.mp rfl
```

```
      have h₂ : 0 ∉ Finset.Ico 1 (k + 1) := by
        intro h₂
        rw[Finset.mem_Ico] at h₂
        linarith
      obtain h₃ := (Finset.erase_eq_iff_eq_insert h₁ h₂).mp rfl
      rw[h₃, Finset.sum_insert h₂]
      unfold choose
      simp
      norm_cast
      have : k.factorial / k.factorial = 1 := Nat.div_self
(Nat.factorial_pos k)
      rw[this]
      simp
```

The next step has a similar idea, but is far less complex, as Mathlib contains a result (`Finset.sum_Ico_succ_top`) allowing us to painlessly split the $i = k$ case from the summation (given k is greater than or equal to zero, which we know via `Nat.zero_le k`).

```
    _ = a ^ (k + 1) + Σ i ∈ Finset.Ico 1 (k + 1), a ^ (k + 1 - i)
* b ^ i * choose k i +
      (Σ i ∈ Finset.Ico 0 k, a ^ (k - i) * b ^ (i + 1) * choose k
i + b ^ (k + 1)) := by
      rw [Finset.range_eq_Ico]
      nth_rw 2 [Finset.sum_Ico_succ_top]
      · unfold choose
        simp
        norm_cast
        have : k.factorial / k.factorial = 1 := Nat.div_self
(Nat.factorial_pos k)
        rw[this]
        simp
      · exact Nat.zero_le k
```

Subsequently we re-index the second summation as $j = i + 1$. This is not as trivial as rewriting i in terms of j — we must prove that there is a bijection between the two summations. We employ the Mathlib result '`Finset.sum_bij`' and supply it with '`(λ i h ↦ i + 1)`', a mapping that sends each element i (together with the fact `h` — that it belongs to `Finset.Ico 0 k`) to $i + 1$. This begins a nested proof involving four goals:

- For all i in `Finset.Ico 0 k`, $i+1$ is in `Finset.Ico 1 (k + 1)` — This is trivial.

- The map is injective — 'Nat.succ_inj'' is a Mathlib result proving injectivity of our map.
- The map is surjective — The goal is to prove that there exists an element of the domain such that its successor equals i — we choose $i - 1$.
- The summation is preserved — This is trivial, and can actually be solved by simply typing simp.

```
    _ = a ^ (k + 1) + Σ i ∈ Finset.Ico 1 (k + 1), a ^ (k + 1 - i)
  * b ^ i * choose k i +
        (Σ j ∈ Finset.Ico 1 (k + 1), a ^ (k + 1 - j) * b ^ j *
  choose k (j - 1) +
        b ^ (k + 1)) := by
      nth_rw 2 [Finset.sum_bij (λ i h ↦ i + 1)]
      · intro i h₁
        rw[Finset.mem_Ico]
        simp
        obtain h₂ := Finset.mem_Ico.mp h₁
        exact h₂.right
      · intro i h₁ j h₂ h₃
        exact Nat.succ_inj'.mp h₃
      · intro i h₁
        simp
        use i - 1
        rw[Finset.mem_Ico] at h₁
        rcases h₁ with ⟨h₁, h₂⟩
        constructor
        · exact Nat.sub_lt_right_of_lt_add h₁ h₂
        · exact Nat.sub_add_cancel h₁
      · simp
```

The next step involves some simple algebraic manipulation. We use Finset.sum_add_distrib to combine the summations into one.

```
    _ = a ^ (k + 1) + Σ i ∈ Finset.Ico 1 (k + 1), a ^ (k + 1 - i)
  * b ^ i * (choose k i + choose k (i - 1)) + b ^ (k + 1) := by
      rw[← add_assoc]
      nth_rw 2 [add_assoc]
      rw[← Finset.sum_add_distrib]
      rw[Finset.sum_congr rfl]
      intro i h₁
```

```
      rw[← mul_add]
```

Now Pascal's Rule (Theorem 3.3) can be applied.

```
    _ = a ^ (k + 1) + Σ i ∈ Finset.Ico 1 (k + 1), a ^ (k + 1 - i)
  * b ^ i * choose (k + 1) i + b ^ (k + 1) := by
      rw[Finset.sum_congr rfl]
      intro i h₁
      rw[Finset.mem_Ico] at h₁
      rcases h₁ with ⟨h₁, h₂⟩
      rw[my.pascals_rule h₁ (Nat.le_of_lt_succ h₂)]
```

Past this point, the remaining steps are all composed of basic manipulation of the expressions and already seen ideas. We leave it as an exercise to study these steps.

```
    _ = Σ i ∈ Finset.Ico 0 (k + 1), a ^ (k + 1 - i) * b ^ i *
  choose (k + 1) i + b ^ (k + 1) := by
      have h₁ : Σ i ∈ Finset.Ico 0 (k + 1), a ^ (k + 1 - i) * b ^
  i * choose (k + 1) i = a ^ (k + 1) * b ^ 0 * choose (k + 1) 0 +
  Σ i ∈ Finset.Ico 1 (k + 1), a ^ (k + 1 - i) * b ^ i * choose (k
  + 1) i := by
        exact rfl
      rw[pow_zero, mul_one] at h₁
      have h₂ : choose (k + 1) 0 = 1 := by
        unfold choose
        simp
        norm_cast
        exact Nat.div_self (Nat.factorial_pos (k + 1))
      rw[h₂, mul_one] at h₁
      rw[← h₁]
    _ = Σ i ∈ Finset.Ico 0 (k + 2), a ^ (k + 1 - i) * b ^ i *
  choose (k + 1) i := by
      have h₁ : Σ i ∈ Finset.Ico 0 (k + 2), a ^ (k + 1 - i) * b ^
  i * choose (k + 1) i = (Σ i ∈ Finset.Ico 0 (k + 1), a ^ (k + 1
  - i) * b ^ i * choose (k + 1) i) + a ^ (k + 1 - (k + 1)) * b ^
  (k + 1) * choose (k + 1) (k + 1) := by
        rw[Finset.sum_Ico_succ_top (Nat.le_add_left 0 (k + 1))]
      rw[Nat.add_sub_add_left] at h₁
      rw[pow_zero, one_mul] at h₁
      have h₂ : choose (k + 1) (k + 1) = 1 := by
        unfold choose
        simp
```

```
          norm_cast
          exact Nat.div_self (Nat.factorial_pos (k + 1))
        rw[h₂, mul_one] at h₁
        rw[← h₁]
      _ = ∑ i ∈ Finset.range (k + 2), a ^ (k + 1 - i) * b ^ i *
    choose (k + 1) i := by
        simp
```

Chapter 4

Prime Numbers

In this chapter we formalise a selection of the results involving prime numbers, including some of the most famous theorems in number theory.

4.1 Introduction to Prime Numbers

We opt to use Mathlib's two definitions of a prime so that Mathlib's elementary prime results are available to us. They are given integer type and natural type respectively. We modify them slightly for clarity before presenting.

The first definition is as follows:

```
def Prime (p : M) : Prop :=
  p ≠ 0 ∧ ¬IsUnit p ∧ ∀ a b, p ∣ a * b → p ∣ a ∨ p ∣ b
```

where for our purposes, M is the set of integers. The definition is given type '`Prop`', that is, it is a proposition. The proposition itself is given by three conditions: that p is nonzero, p is not a unit, and for all integers a and b, if p divides ab, then p divides a or p divides b. Hence, when we define an integer p to be prime as a starting hypothesis in any result, we can immediately derive that p satisfies all three of the conditions given by `Prime`.

The second definition is given as:

```
def Nat.Prime (p : ℕ) : Prop :=
  Irreducible p
```

Working over the natural numbers, the irreducibility condition is enough to define a prime — but this is essentially the same as the definition over the integers. The reason for utilising both definitions is simple — it is usually more convenient to use one over the other, generally due to the types of the variables in use, or due to required results in Mathlib using it.

The third condition in `Prime` relates to Euclid's Lemma. It is used as a characterisation of a prime number — we aim to prove it for ourselves. The formalisation of Euclid's Lemma is not so obvious, as we need to declare a prime p, however, as we have seen, defining a prime number via Mathlib implicitly considers Euclid's Lemma, therefore creating a cyclic argument. Thus we choose to exploit a generalisation that does not rely on p being defined as prime:

Lemma 4.1. *(Euclid's Lemma) Let p, a and b be integers. If p divides ab and $\gcd(a, p) = 1$ then p divides b.*

Proof. Suppose $p \mid ab$ and $\gcd(a, p) = 1$. By Bézout's Lemma (Theorem 2.13) there exist integers x and y such that $ax + py = 1$. Multiply by b to obtain $axb + pyb = b$. Now rearranging, $(ab)x + (p)yb = b$. Since $p \mid ab$ by assumption, and clearly $p \mid p$, $p \mid (ab)x + (p)yb$, so $p \mid b$. □

The formalisation is rather trivial. We bestow p, a and b with integer type. To summarise, we leverage Bézout's Lemma (recall this supplies integers defined by functions $L(a, b)$ and $R(a, b)$), multiply by b, perform some algebraic manipulation, and conclude accordingly. Notice that we use `Prime` as opposed to `Nat.Prime`, as p, a and b are all integers.

```
lemma euclids_lemma {p a b : ℤ} (h₁ : p ∣ a * b) (h₂ : (a.gcd p : ℤ
    ) = 1) : p ∣ b := by
  rw[← my.bezouts_lemma] at h₂
  have h₃ : (L a p * a + R a p * p) * b = 1 * b := by rw[h₂]
  rw[one_mul, add_mul, mul_assoc, mul_comm] at h₃
  nth_rw 4 [mul_comm] at h₃
  rw[← h₃]
  exact my.dvd_add_of_two_mul h₁ my.dvd_mul_right
```

In a later formalisation involving Euclid's Lemma, we opt to use Mathlib's definition, as it is more useful in the context. This is given (again with small modifications for clarity) as:

```
theorem Prime.dvd_or_dvd {a b p : M} (h : p ∣ a * b) (hp : Prime p)
    : p ∣ a ∨ p ∣ b :=
```

```
hp.2.2 a b h
```

Of course since Euclid's Lemma is given by the definition of a prime, it is only necessary to point towards it in the formalisation. Therefore, '`hp.2.2`' is written, referring to the right-hand side of the right-hand side conjunction in the `Prime` definition, i.e. for a conjunction $A \wedge B \wedge C = A \wedge (B \wedge C)$, `.2.2` indicates C.

Euclid's Lemma is critical in the proof of the following lemma (which we need for the final formalisation of Chapter 5). For the remainder of this chapter, define k, n and p to be of natural number type.

Lemma 4.2. *Let p be a prime and $0 < k < p$. Then p divides $\binom{p}{k}$.*

Proof. By Lemma 3.2 it is easy to see that $p \mid k \cdot \binom{p}{k}$. Thus by Lemma 4.1 $p \mid k$ or $p \mid \binom{p}{k}$. Suppose for contradiction that p does not divide $\binom{p}{k}$. Then $p \mid k$ so $k \geq p$. But $k < p$, so we have a contradiction. Hence $p \mid \binom{p}{k}$. □

The formalisation is simple, the only real problems coming from casting variables between types, as seen already in Chapter 3.

```
lemma prime_dvd_choose (h₁ : Prime (p : ℤ)) (h₂ : (k : ℤ) > 0) (h₃
    : (k : ℤ) < (p : ℤ)) : (p : ℤ) ∣ choose p k := by
  have h₄ : (k : ℤ) ≤ (p : ℤ) := le_of_lt h₃
  have h₅ : (p : ℤ) ∣ ((k : ℤ) * choose p k) := by
    use (choose (p - 1) (k - 1))
    norm_cast at h₂ h₄
    exact my.choose_helper h₂ h₄
  apply Prime.dvd_or_dvd h₁ at h₅
  by_contra contra
  obtain h₅ ∣ h₆ := h₅
  · have : (k : ℤ) ≥ (p : ℤ) := by
      exact Int.le_of_dvd h₂ h₅
    linarith
  · exact contra h₆
```

With regard to the definition of a prime number, we can advance further by specifying the exact form every prime number (excluding two and three) must assume.

Theorem 4.3. *Let p be a prime greater than 3. Then there exists a natural number k such that $p = 6k + 1$ or $p = 6k + 5$.*

Proof. Let $p = 6k + r$. We check the cases $r = 0, \ldots, 5$. If $p = 6k + 1$ or $p = 6k + 5$ we are done. Each $p = 6k + r$, $r = 0, 2, 3, 4$ can be shown to be composite, and thus are not prime. □

Initially it is necessary to show that we can indeed write p as $6k + r$. This can be done by adopting the Mathlib result: '`Nat.div_add_mod`', which proposes that for some natural numbers m and n, `n * (m / n) + m % n = m`. Letting $k = \lfloor \frac{p}{6} \rfloor$ and $r = p \bmod 6$ and applying this equation it is clear that $p = 6k + r$.

```
theorem form_of_primes (h₁ : Nat.Prime p) (h₂ : p > 3) : ∃ k, p =
    6*k+1 ∨ p = 6*k+5 := by
  let k := p / 6
  let r := p % 6
  have h₃ : p = 6 * k + r := (Nat.div_add_mod p 6).symm
```

We must also confirm that r is less than 6 — a quite obvious fact, but one which must be made clear for the next step to be successful. We take advantage of the Mathlib result '`Nat.mod_lt`', which tells us that `n > 0 → m % n < n`. Pairing it with `apply` (noting that $m = 0, n = 6$), it remains to verify that six is greater than zero, which can be cleared with `linarith`.

```
  have h₄ : r < 6 := by
    apply Nat.mod_lt p
    linarith
```

Now we can begin a case analysis on r. We deploy the `match` tactic, which sets a new goal: to check $6k+r$ for $r = 0, 1, \ldots, 5$. The structure of the remainder of this formalisation is subtly different to anything seen until this point, but is straightforward to understand at a glance. Essentially, we manually write each possible case (as opposed to the bullet point approach seen for previous sets of subgoals) and then evaluate the case below. As, an example, see the case $r = 0$.

```
  match r with
  | 0 =>
```

Below this we evaluate $p = 6k + 0 = 6k$. Clearly this is composite as $6k = 2 \cdot 3k$. Thus it is imperative to utilise `exfalso`. It is easy to obtain $p = 2 \cdot 3k$ using a `have` statement with `linarith`. This can be substituted into the hypothesis supposing that p is prime ($\mathtt{h}_1$). Next we apply `Nat.not_prime_mul` to this hypothesis, which states that no prime can be of the form

ab for some a and b both not equal to one. This has two effects: Firstly, the hypothesis converts to `False`, which is clearly the goal due to use of `exfalso`, so using `assumption` clears the main goal. Despite this, there are now two other goals: to prove that $a = 2 \neq 1$ and $b = 3k \neq 1$. These are both solvable by `linarith`, and we use a new tactic, `all_goals` (which applies the specified tactic to all available goals), to clear them both simultaneously.

```
    exfalso
    have : p = 2 * (3 * k) := by linarith
    rw[this] at h₁
    apply Nat.not_prime_mul at h₁
    assumption
    all_goals linarith
```

Next we evaluate the case $r = 1$. The goal is as follows: `∃ k, p = 6 * k + 1 ∨ p = 6 * k + 5`. Writing `use k` removes the existential quantifier, `left` focuses on the left-hand side of the goal, and since the left-hand side of the goal is $p = 6k + 1$, this is true by assumption.

```
  | 1 =>
    use k
    left
    assumption
```

For the remaining cases ($r = 2, 3, 4, 5$) the respective arguments are almost identical.

```
  | 2 =>
    exfalso
    have : p = 2 * (3 * k + 1) := by linarith
    rw[this] at h₁
    apply Nat.not_prime_mul at h₁
    assumption
    all_goals linarith
  | 3 =>
    exfalso
    have : p = 3 * (2 * k + 1) := by linarith
    rw[this] at h₁
    apply Nat.not_prime_mul at h₁
    assumption
    all_goals linarith
  | 4 =>
```

```
    exfalso
    have : p = 2 * (3 * k + 2) := by linarith
    rw[this] at h₁
    apply Nat.not_prime_mul at h₁
    assumption
    all_goals linarith
  | 5 =>
    use k
    right
    assumption
```

4.2 Euclid's Theorem

Continuing our discussion on primes, we derive a result that seems intuitively apparent, but requires a rigorous argument.

Lemma 4.4. *Let n be a natural number greater than or equal to two. Then there exists a prime p such that p divides n.*

Proof. We proceed by strong induction on n. Assume that for all $m < n$, $m \geq 2$ implies that there exists p such that $p \mid m$ and p is prime. If n is prime then $n \mid n$ so we are done, hence suppose that n is not prime. Then n must be divisible by some l, $2 \leq l < n$. But by the inductive hypothesis, since $l < n$ and $l \geq 2$, there exists a prime number, say q, such that $q \mid l$. Thus $l = qk$ for some $k \in \mathbb{N}$. Therefore, since $l \mid n$, $qk \mid n$ so $q \mid n$. This is a prime number dividing n, so there exists a prime number dividing n for any n. □

First we highlight the proposition statement in Lean — since we cannot explicitly declare with the existential quantifier that the existing p is prime, we must use a conjunction to secure the primality condition.

```
lemma every_nat_has_prime_factor (h₁ : n ≥ 2) : ∃ p, p ∣ n ∧
    Nat.Prime p := by
```

We use the `induction'` tactic as usual for an inductive proof, but with the addition of the `using` condition to specify that we require strong induction. For convenience the induction variable is set to n.

```
  induction' n using Nat.strong_induction_on with n ih
```

Since two cases are being considered, it is natural to utilise `by_cases`. The first case is trivial — given that we are aware of a prime number dividing n, `use` can be adopted.

```
by_cases h₂ : Nat.Prime n
· use n
```

It remains to prove that n has a prime divisor if it is not prime. As a consequence of compositeness, the Mathlib result '`Nat.not_prime_iff_exists_dvd_lt`' elaborates on $\mathtt{h_1}$ to declare that there exists a natural number that is less than n, divides n and is greater than or equal to two. From this we can formally define an arbitrary integer l with these three separate conditions using `rcases`.

```
· rw [Nat.not_prime_iff_exists_dvd_lt h₁] at h₂
  rcases h₂ with ⟨l, h₂, h₃, h₄⟩
```

Recall the inductive hypothesis: 'for all $m < n$, $m \geq 2$ implies that there exists p such that $p \mid m$ and p is prime'. The natural number l is less than n and is greater than or equal to two, so there must be exist a prime p dividing m. This fact can be acquired with `obtain`.

```
  obtain h₅ := ih l h₄ h₃
```

Similar to the previous use of `rcases`, we apply it to $\mathtt{h_5}$ to get a number q such that q divides l (this is the new $\mathtt{h_5}$) and q is prime.

```
  rcases h₅ with ⟨q, h₅, h₆⟩
```

Once again we use `rcases`, this time on the new $\mathtt{h_5}$, to get $l = qk$ for some natural number k. Then, substituting into $\mathtt{h_5}$ we get qk divides n, and so by Lemma 2.5, q divides n. Hence q satisfies the goal: that there is some prime divisor of n.

```
  rcases h₅ with ⟨k, h₇⟩
  rw[h₇] at h₂
  obtain h₈ := my.prod_left_dvd h₂
  use q
```

Lemma 4.4 is vital in our analysis of a well-known theorem known as Euclid's Theorem, which reveals an interesting property about the prime numbers.

Theorem 4.5. *(Euclid's Theorem) There are infinitely many primes.*

Proof. Let $n \in \mathbb{N}$ and $m = n! + 1$. Clearly $m \geq 2$ as $n! \geq 1$, so we can apply Lemma 4.4 — there exists a natural number, say q, such that q divides m and q is prime.

We can prove that q does not divide $n!$. Suppose $q \mid n!$ — then $q \mid m - n!$. But $m - n! = n! + 1 - n! = 1$ so $q \mid 1$, and thus $q = 1$. Yet since q is prime, $q \geq 2$, so by contradiction, q does not divide $n!$.

As q does not divide $n!$, it follows that $q > n$, as if $q \leq n$, then $q \mid n!$, a contradiction. Hence, q is a prime greater than n, so for every n there is a prime greater than n, i.e. there is no largest prime, so there are infinitely many. □

The proposition statement follows in a similar manner as seen in Lemma 4.4.

```
theorem euclids_theorem : ∃ p, Nat.Prime p ∧ p > n := by
```

The `let` tactic allows us to establish that $m = n! + 1$.

```
  let m := n.factorial + 1
```

We prove that m is greater than or equal to two. The tactic `unfold` is required to substitute the value of m into the inequality. By Mathlib's `Nat.one_le_iff_ne_zero`, which states that a natural number is greater than or equal to one if and only if it is nonzero, it is clear that $n!$ is greater than or equal to one. Thus it follows that $n! + 1$ is greater than or equal to two.

```
  have h₁ : m ≥ 2 := by
    unfold m
    obtain h₁ := Nat.one_le_iff_ne_zero.mpr (Nat.factorial_ne_zero n)
    linarith
```

Now we employ `obtain` to get that (since m is greater than or equal to two) there is a natural number (which we call q) such that q divides m ($\mathtt{h}_2$) and q is prime ($\mathtt{h}_3$).

```
  obtain ⟨q, h₂, h₃⟩ := every_nat_has_prime_factor h₁
```

Next it is demonstrated that q does not divide $n!$. Since the goal contains a negation, beginning with `intro` removes the negation and changes the goal to `False`, establishing the non-negated version of the goal as an assumption.

This is identical to how the `by_contra` tactic operates — either approach works here.

```
have h₄ : ¬ q ∣ n.factorial := by
  intro h₅
```

We utilise the `apply` tactic on $\mathtt{h_5}$ (together with `dvd_sub`, a result formalised in the Appendix), which alters $\mathtt{h_5}$ to instead assert that q divides $m - n!$. From this it is straightforward to conclude that q equals one by substituting m into $\mathtt{h_5}$ and using `simp` (which conveniently does all the work for us).

```
  apply my.dvd_sub h₂ at h₅
  unfold m at h₅
  simp at h₅
```

But q is greater than or equal to two by `Nat.Prime.two_le` (a result stating that primes are greater than or equal to two). Hence a contradiction arises, detected by `linarith`.

```
  have h₆ : q ≥ 2 := Nat.Prime.two_le h₃
  linarith
```

Now we show that q is greater than n by looking for a contradiction. If q is less than or equal to n (which we acquire through the usage of `by_contra` and `push_neg`) then by `Nat.dvd_factorial` ($q > 0$ and $q \leq n$ then $q \mid n!$), since q is also greater than zero (`Nat.Prime.pos` proves that any prime is positive), clearly q divides $n!$. But this contradicts our previous demonstration that q does not divide $n!$, so q is greater than n.

```
have h₅ : q > n := by
  by_contra h₆
  push_neg at h₆
  apply Nat.dvd_factorial (Nat.Prime.pos h₃) at h₆
  exact h₄ h₆
```

Notice that the goal has not been altered yet during this proof, so it remains as: there exists some natural number p such that p is prime and p is greater than n. We have verified that q satisfies this, so we are done.

```
use q
```

4.3 Fundamental Theorem of Arithmetic

In Lemma 4.4 we recognised that every natural number greater than or equal to two has a prime factor. The Fundamental Theorem of Arithmetic extends this fact to something far more interesting.

Theorem 4.6. *(Fundamental Theorem of Arithmetic) Let n be a natural number greater than or equal to two. Every n can be represented uniquely (up to the ordering of the factors) as a product of prime numbers.*

The proof of this theorem is split into proving the existence of such a prime factorisation, and then proving that this prime factorisation is unique. Proving existence is significantly easier in the context of Lean.

Lemma 4.7. *Let n be a natural number greater than or equal to two. Every n can be represented as a product of prime numbers.*

Proof. We prove that there exists a set of prime numbers (with duplicates) such that the collective product of all elements in the set is equal to n. We proceed by strong induction on n. Our inductive hypothesis is that for all $m < n$, if $m \geq 2$ then there exists a set S of primes (with duplicates) such that m equals the product of all the elements in the set. If n is prime, then it follows trivially that n is a product of primes, thus it suffices to only check the case in which n is not prime.

Since n is not prime, there exists a natural number, say k, such that $2 \leq k < n$ and $k \mid n$. Hence $n = kl$ for some natural number l such that $2 \leq l < n$. By the inductive hypothesis there must be sets, say S_1 and S_2, of primes such that the product of all the elements in each set is equal to k and l respectively — so k and l can be written as products of primes. Since $n = kl$ and k and l can be written as products of primes, it follows that n can be written as a product of primes (the set of primes being the collective set of elements of both S_1 and S_2). □

Notice the requirement to allow for repeated primes in our set of primes — this renders the `Finset` structure obsolete. Instead, we employ the `Multiset` structure, which is simply a finite set, but without any restriction on duplicate entries. The `Multiset` library contains a useful definition, `prod`, defining the product of all elements of a given multiset. These tools provide the framework for proposition written in Lean.

```
lemma ftoa_existence (h₁ : n ≥ 2) : ∃ (S : Multiset ℕ), (∀ p ∈ S,
    Nat.Prime p) ∧ n = S.prod := by
```

We proceed by strong induction with inductive hypothesis `ih`, and quickly prove the proposition for if n is prime, using the set $\{n\}$ as the example of a set of primes such that n is equal to the product of the elements. The `simp` tactic drastically simplifies the goal for us, leaving the simple task of verifying that n is prime, which is of course true by assumption.

```
induction' n using Nat.strong_induction_on with n ih
by_cases case : Nat.Prime n
· use {n}
  simp
  assumption
```

It remains to establish the validity of the proposition for composite n. The Mathlib result '`Nat.not_prime_iff_exists_dvd_lt`' helps establish that since n is not prime, and n is greater than or equal to two (by $\mathtt{h_1}$), there must exist a natural number dividing n that is greater than or equal to two and less than n. We use `obtain` to define k to be such a number, and to establish separate hypotheses for each of the given statements. Since k divides n, we can further simplify with `rcases`, defining a natural number l such that $n = kl$. It is easy to see that l must be greater than or equal to two and less than n.

```
· rw[Nat.not_prime_iff_exists_dvd_lt h₁] at case
  obtain ⟨k, h₂, h₃, h₄⟩ := case
  rcases h₂ with ⟨l, h₂⟩
  have h₅ : l ≥ 2 := by
    rw[h₂] at h₄
    exact one_lt_of_lt_mul_right h₄
  have h₆ : l < n := by
    rw[h₂]
    rw[mul_comm]
    have : l > 0 := by linarith
    exact (Nat.lt_mul_iff_one_lt_right this).mpr h₃
```

Since k and l are natural numbers greater than or equal to two, we can apply the inductive hypothesis to each, to establish that there exists multisets of primes with the products of the elements equal to k and l. We utilise `rcases` to define two such multisets as S_1 (for k) and S_2 (for l).

```
  obtain h₇ := ih k h₄ h₃
  obtain h₈ := ih l h₆ h₅
  rcases h₇ with ⟨S₁, h₇⟩
  rcases h₈ with ⟨S₂, h₈⟩
```

Due to the approach taken, h_7 and h_8 both contain conjunctions, with the first conjunct being that each element of the set is prime, and the second being that the product of the elements of the set is equal to the required variable. As such, we use `obtain` to split them into separate hypotheses.

```
obtain ⟨h9, h10⟩ := h7
obtain ⟨h11,h12⟩ := h8
```

Now we have the sufficient knowledge to solve the goal. We deploy `rw` to rewrite the goal in terms of the `prod` of S_1 and S_2. It remains to prove that there does indeed exist a multiset of natural numbers, say S, such that each element is prime and S_1`.prod*`S_2`.prod=S.prod`. We suppose that the set $S_1 + S_2$, which defines the set of all elements (with duplicates) in both S_1 and S_2, satisfies this criteria.

```
rw[h2, h10, h12]
use S1 + S2
```

We prove that each element of S_1+S_2 is prime, and that S_1`.prod * `S_2`.prod = (`S_1` + `S_2`).prod`. First, define a natural number p and suppose p is an element of $S_1 + S_2$. Then by `Multiset.mem_add`, p is in S_1 or p is in S_2. We check each case with `obtain`. If p is in S_1 then by an earlier hypothesis p is prime. If p is in S_2 then again from earlier work we know p is prime. Thus every element of $S_1 + S_2$ is prime.

```
constructor
· intro p hp
  rw [Multiset.mem_add] at hp
  obtain hp1 | hp2 := hp
  · exact h9 p hp1
  · exact h11 p hp2
```

Finally, by a Mathlib result it is trivial to show that S_1`.prod * `S_2`.prod = (`S_1` + `S_2`).prod`. This completes the formalisation.

```
· rw[Multiset.prod_add]
```

Now we will work towards proving the 'uniqueness' part of Theorem 4.6. This is more involved than the 'existence' part, and thus we begin with some preliminary results. We begin with a form of Euclid's Lemma in the context of multisets.

Lemma 4.8. *Let k be a natural number, p be prime, and S a multiset of natural numbers. If p divides the product of all the elements in S, then there is some element of S, say x, such that p divides x.*

Proof. A subtle change is applied to the lemma for this proof — we suppose p divides the product of the elements of the set $\{k\} + S$ for some natural number k (where for two multisets A and B, we write $A + B$ to refer to the merging of the two sets) and prove that there exists an element in $\{k\} + S$ divisible by p. This change is required for a rigorous Lean proof, as will be explained later.

We proceed by induction on S. For the base case, set $S = \varnothing$. Then by assumption, p divides the product of $\{k\} + \varnothing = \{k\}$, i.e. $p \mid k$. Hence there does indeed exist an element in $\{k\} + \varnothing = \{k\}$ divisible by p.

Proceeding with the inductive step, we assume the inductive hypothesis — that if p divides the product of the elements of $\{k\} + S$, then there exists an element x in $\{k\} + S$ divisible by p — is true. We aim to prove that for some set with one additional element, say $\{k\} + \{l\} + S$, if p divides the product of all elements in this set, then p divides some element of this set. Let $\text{prod}(S)$ denote the product of the elements of S. Then we can write the product of the elements of $\{k\} + \{l\} + S$ as $l \cdot (\text{prod}(S) \cdot k)$ and therefore, by Euclid's Lemma, $p \mid l$ or $p \mid \text{prod}(S) \cdot k$. If $p \mid l$ then clearly l is an element in $\{k\} + \{l\} + S$ that is divisible by p. If $p \mid \text{prod}(S) \cdot k$, then by the inductive hypothesis, there exists an element x in $\{k\} + S$ divisible by p. Clearly x is in $\{k\} + \{l\} + S$, and so it satisfies the required criteria. □

Note that `Multiset.cons k S` refers to $\{k\} + S$ and `(Multiset.cons k S).prod` refers to the product of the elements of the set `Multiset.cons k S`. We proceed by induction on S with the `induction'`, specifying that we require induction on a multiset (which is defined in Mathlib via `Multiset.induction_on`). The necessity of beginning with a set $\{k\} + S$ rather than simply a set S comes from how induction works in Lean. Whenever the `induction'` tactic is called, it forces a base case which sets the induction variable to zero (or in this case, the empty set) — this cannot be changed. However, it is easy to see that the base case would not hold if S were empty, as clearly there would be no elements in S that could be divisible by p. Thus we require S to contain at least one element, which can be established by merging S with some arbitrary natural number k.

```
lemma prime_dvd_multiset_element (h₁ : Nat.Prime p) (h₂ : p ∣
    (Multiset.cons k S).prod) : ∃ x ∈ Multiset.cons k S, p ∣ x := by
  induction' S using Multiset.induction_on with l S ih
```

Proving the base case is trivial.

```
  · simp at *
    assumption
```

Proceeding with the inductive step, we first modify `h₂` (which now involves the set $\{l\}$) to obtain `l * (S.prod * k)`. This involves Mathlib's `Multiset.prod_cons`, which allows us to rewrite '`Multiset.cons a A`' as '`a*A.prod`' for some element a and multiset A, followed by some simple applications of multiplicative properties. Finally we invoke `Nat.Prime.dvd_mul`, the version of Euclid's Lemma specifically for natural numbers, to obtain that p divides l or p divides $\text{prod}(S) \cdot k$.

```
  · rw [Multiset.prod_cons, Multiset.prod_cons, mul_comm, mul_assoc,
    Nat.Prime.dvd_mul h₁] at h₂
```

Employing `obtain` we can analyse both cases. If p divides l then it is trivial to prove that there exists an element in $\{k\} + \{l\} + S$ divisible by p (as l is this element). Suppose instead that p divides $\text{prod}(S) \cdot k$. Then p divides $\text{prod}(\{k\} + S)$ by `Multiset.prod_cons`. Therefore we can apply the inductive hypothesis, and via some simple manipulation and use of `Multiset.cons_swap` (which allows for $\{k\} + \{l\} + S$ to become $\{l\} + \{k\} + S$) and `Multiset.mem_cons_of_mem` (which states that the element x is in $\{l\} + \{k\} + S$ as it is in $\{k\} + S$) we are done.

```
    obtain h₂ ∣ h₃ := h₂
    · use l
      constructor
      · simp
      · assumption
    · rw[mul_comm, ← Multiset.prod_cons] at h₃
      apply ih at h₃
      rcases h₃ with ⟨x, h₃, h₄⟩
      use x
      constructor
      · rw[Multiset.cons_swap]
        exact Multiset.mem_cons_of_mem h₃
      · assumption
```

Another result necessary for the formalisation of the Fundamental Theorem of Arithmetic is the following:

Lemma 4.9. *There does not exist a nonempty set of primes such that the product of all elements in the set equals 1.*

Proof. We prove that if there is a multiset S such that $\text{prod}(S) = 1$ and every element of S is prime, then $S = \varnothing$. Suppose for contradiction that $S \neq \varnothing$. Then there exists some natural number, say p, in S, and p must be prime by assumption — therefore $p \geq 2$. Clearly $S = \{p\} + (S \setminus \{p\})$ and so $\text{prod}(S) = \text{prod}(\{p\} + (S \setminus \{p\})) = p \cdot \text{prod}(S \setminus \{p\}) = 1$ (by assumption). Hence $p = 1$ and $\text{prod}(S \setminus \{p\}) = 1$. But $p \geq 2$, so we have a contradiction. □

The formalisation is straightforward, and so it is left as an exercise for the reader to analyse it.

```
lemma prod_primes_eq_one_empty (h₁ : S.prod = 1) (h₂ : ∀ p ∈ S,
    Nat.Prime p) : S = ∅ := by
  by_contra h₃
  apply Multiset.exists_mem_of_ne_zero at h₃
  rcases h₃ with ⟨p, h₃⟩
  obtain h₄ := Nat.Prime.two_le (h₂ p h₃)
  have h₅ : S.prod = p * (S.erase p).prod := (Multiset.prod_erase
    h₃).symm
  rw[h₅] at h₁
  apply mul_eq_one.mp at h₁
  linarith
```

We now possess the necessary lemmas to proceed with the proof of the 'uniqueness' of the Fundamental Theorem of Arithmetic. The proof resembles the traditional approach, but is adjusted to accommodate for some of Lean's limitations, specifically concerning minimality.

Lemma 4.10. *Let n be a natural number greater than or equal to two. If we write two prime factorisations for n, then these prime factorisations are the same up to the ordering of the factors.*

Proof. We prove that if there exists two sets of primes (with duplicates), say S_1 and S_2, such that the products of the elements of the respective sets are both equal to n, then $S_1 = S_2$. We proceed by strong induction, with inductive hypothesis being that for all $m < n$, if $m \geq 2$, and if for any two sets of primes S_1 and S_2 we have both products of the elements in the sets

are equal to n, then $S_1 = S_2$.

Clearly S_1 is nonempty as $\text{prod}(S_1) = n \geq 2$. So there is some prime p in S_1. Define $S_1' = S_1 \setminus \{p\}$. Then $S_1 = S_1' + \{p\}$ and $n = \text{prod}(S_1) = \text{prod}(S_1' + \{p\}) = p \cdot \text{prod}(S_1')$ so $p \mid n$. But $n = \text{prod}(S_2)$ so $p \mid \text{prod}(S_2)$.

Similar to S_1, S_2 is nonempty, so there is an element a in S_2. Since $p \mid \text{prod}(S_2)$, $p \mid \text{prod}(\{a\} + S_2 \setminus \{a\})$ so by Lemma 4.8 there is element q of $\{a\} + S_2 \setminus \{a\}$ such that $p \mid q$ (recall that in Lean Lemma 4.8 requires an input of a singleton set together with a complete multiset). Further we know that q is prime, thus it follows that $p = q$, as p is prime so cannot equal 1.

Let $S_2' = S_2 \setminus \{q\}$, so $S_2 = \{q\} + S_2'$. We have $n = \text{prod}(S_2) = q \cdot \text{prod}(S_2') = p \cdot \text{prod}(S_2')$. Hence $\frac{n}{p} = \text{prod}(S_2')$, and also $\frac{n}{p} = \text{prod}(S_1')$ as $n = p \cdot \text{prod}(S_1')$. Suppose $\frac{n}{p} = 1$. Then $\text{prod}(S_1') = \text{prod}(S_2') = 1$. By Lemma 4.9, since S_1' and S_2' are multisets of primes, $S_1' = \varnothing$ and $S_2' = \varnothing$. Hence, as $p = q$ we have $\{p\} + \varnothing = \{q\} + \varnothing$, so $\{p\} + S_1' = \{q\} + S_2'$, i.e. $S_1 = S_2$.

Now suppose $\frac{n}{p} \neq 1$. It is easy to see that $2 \leq \frac{n}{p} < n$, and so we can apply the inductive hypothesis to get that $S_1' = S_2'$. Therefore, again $S_1 = S_2$, and so for any $n \geq 2$ we have that the multisets of primes producing a factorisation of n are equal — that is, that the factorisation into primes is unique up to the ordering of the factors. □

Note that due to the length and complexity of this proof, we generally refrain from using generic hypothesis names. We begin by declaring our intent to perform strong induction, and unpack the proposition into various variables and hypotheses.

```
lemma ftoa_uniqueness (h₁ : n ≥ 2) : ∀ (S₁ S₂ : Multiset ℕ), (∀ p ∈
    S₁, Nat.Prime p) ∧ n = S₁.prod → (∀ p ∈ S₂, Nat.Prime p) ∧ n =
    S₂.prod → S₁ = S₂ := by
  induction' n using Nat.strong_induction_on with n ih
  intro S₁ S₂ hS₁ hS₂
  rcases hS₁ with ⟨hS₁_prime, hS₁_eq⟩
  rcases hS₂ with ⟨hS₂_prime, hS₂_eq⟩
```

We prove S_1 is nonempty, by supposing for contradiction that it is empty (via `intro`) and deriving that $n = 1$, using `Multiset.empty_eq_zero` — that $\varnothing = 0$ (this is required simply due to the notation used in the next Mathlib result) — and `Multiset.prod_zero` — that $\text{prod}(0) = 1$.

```
have S₁_nonempty : S₁ ≠ ∅ := by
  intro h
  rw[h, Multiset.empty_eq_zero, Multiset.prod_zero] at hS₁_eq
  linarith
```

Since S_1 is nonempty, it follows that there is some natural number (say p) such that p is an element of S_1. By `hS₁_prime` every element of S_1 is prime, and so p must be prime.

```
obtain ⟨p, hp⟩ := Multiset.exists_mem_of_ne_zero S₁_nonempty
have p_prime : Nat.Prime p := hS₁_prime p hp
```

We create a new variable, S_1', using `let`. By `Multiset.cons_erase`, since p is in S_1, $\{p\} + S_1 \setminus \{p\} = S_1$, and so we can obtain $n = p \cdot \text{prod}(S_1')$. It follows from `Dvd.intro` (for some a, b and c, if $ac = b$ then $a \mid b$) that p divides n, and hence p divides $\text{prod}(S_2)$.

```
let S₁' := S₁.erase p
have S₁'_eq : S₁ = Multiset.cons p S₁' := (Multiset.cons_erase
  hp).symm
have n_eq : n = p * (S₁').prod := by
  rw[hS₁_eq, S₁'_eq]
  simp
have p_dvd_n : p ∣ n := Dvd.intro (S₁').prod n_eq.symm
have p_dvd_S₂_prod : p ∣ S₂.prod := by
  rw [hS₂_eq] at p_dvd_n
  exact p_dvd_n
```

Now we move to S_2. By the same argument as seen for S_1, S_2 is nonempty, and so there is some natural number, say a, such that a is in S_2. Hence by `Multiset.cons_erase` we can write '$p \mid \text{prod}(S_2)$' as '$p \mid \text{prod}(\{a\} + S_2 \setminus \{a\})$'.

```
have S₂_nonempty : S₂ ≠ ∅ := by
  intro h
  rw[h, Multiset.empty_eq_zero, Multiset.prod_zero] at hS₂_eq
  linarith
obtain ⟨a, ha⟩ := Multiset.exists_mem_of_ne_zero S₂_nonempty
rw[← Multiset.cons_erase ha] at p_dvd_S₂_prod
```

Lemma 4.8 can be applied to obtain some q such that q is in $\{a\} + S_2 \setminus \{a\}$ and p divides q. We rewrite '$q \in \{a\} + S_2 \setminus \{a\}$' as '$q \in S_2$', deduce that q is

prime and derive from `Nat.prime_dvd_prime_iff_eq` that $p = q$.

```
obtain ⟨q, hq, p_dvd_q⟩ := my.prime_dvd_multiset_element p_prime
  p_dvd_S₂_prod
rw[Multiset.cons_erase ha] at hq
have q_prime : Nat.Prime q := hS₂_prime q hq
have p_eq_q : p = q := (Nat.prime_dvd_prime_iff_eq p_prime
  q_prime).mp p_dvd_q
```

We define S_2' and prove its relation to S_2 analogously to S_1, and easily show that $n = p \cdot \text{prod}(S_2')$. Since p is greater than 0 (noting that every prime is positive — `Nat.Prime.pos`) we can divide by p and thus obtain two equations: $\frac{n}{p} = \text{prod}(S_1')$ and $\frac{n}{p} = \text{prod}(S_2')$.

```
let S₂' := S₂.erase q
have S₂'_eq : S₂ = Multiset.cons q S₂' := (Multiset.cons_erase
  hq).symm
have n_eq' : n = p * S₂'.prod := by
  rw[hS₂_eq, S₂'_eq]
  rw[p_eq_q]
  simp
have h₂ : p > 0 := Nat.Prime.pos p_prime
have frac_eq : n / p = S₁'.prod := by
  rw [n_eq]
  exact Nat.mul_div_right S₁'.prod h₂
have frac_eq' : n / p = S₂'.prod := by
  rw[n_eq']
  exact Nat.mul_div_right S₂'.prod h₂
```

Now we check the case $\frac{n}{p} = 1$ and $\frac{n}{p} \neq 1$, using `by_cases`. For the case $\frac{n}{p} = 1$, we prove that S_1' and S_2' must be empty, by applying Lemma 4.9, and then verifying that every element of each S_i' is prime (and thus showing that Lemma 4.9 does indeed apply). This verification follows by assumption after noting that for all r in S_i', r is in S_i (`Multiset.mem_of_mem_erase`). Thus, rewriting the goal ($S_1 = S_2$) and simplifying, we obtain $p = q$, which we know to be true.

```
by_cases case₂ : n / p = 1
· rw[case₂] at frac_eq frac_eq'
  symm at frac_eq frac_eq'
  have S₁'_empty : S₁' = ∅ := by
    apply my.prod_primes_eq_one_empty frac_eq
    intro r hr
```

```
    apply Multiset.mem_of_mem_erase at hr
    exact hS₁_prime r hr
  have S₂'_empty : S₂' = ∅ := by
    apply my.prod_primes_eq_one_empty frac_eq'
    intro r hr
    apply Multiset.mem_of_mem_erase at hr
    exact hS₂_prime r hr
  rw [S₁'_eq, S₁'_empty, S₂'_eq, S₂'_empty]
  simp
  exact p_eq_q
```

Now we move on to the other case, that $\frac{n}{p} \neq 1$. This is displayed as '¬n/p=1' and so the `push_neg` tactic is employed to express this in standard form. We prove via an array of inequality-based Mathlib results that $\frac{n}{p}$ is greater than or equal to two and less than n; it is left as an exercise for the reader to decode this.

```
· push_neg at case₂
  have frac_gr_one : n / p ≥ 2 := by
    have : n / p ≠ 0 := by
      apply Nat.div_ne_zero_iff.mpr
      constructor
      · exact Nat.ne_zero_of_lt h₂
      · apply Nat.le_of_dvd _ p_dvd_n
        · exact Nat.zero_lt_of_lt h₁
    have : n / p > 0 := Nat.zero_lt_of_ne_zero this
    exact Nat.lt_of_le_of_ne this case₂.symm
  have frac_lt : n / p < n := by
    apply Nat.div_lt_self
    exact Nat.zero_lt_of_lt h₁
    exact Nat.Prime.one_lt (hS₁_prime p hp)
```

It follows from the inductive hypothesis that $S_1' = S_2'$, provided we can prove that every element of each S_i' is prime, and $\frac{n}{p} = \text{prod}(S_i')$ — a fact that is derived quite trivially. Hence we can rewrite the goal using our hypotheses to achieve reflexivity. This completes the formalisation.

```
  have ih_result : S₁' = S₂' := by
    apply ih (n / p) frac_lt frac_gr_one
    constructor
    · intro r hr
      apply hS₁_prime r
      exact Multiset.mem_of_mem_erase hr
```

```
      · exact frac_eq
      constructor
      · intro r hr
        apply hS₂_prime r
        exact Multiset.mem_of_mem_erase hr
      · exact frac_eq'
    rw [S₁'_eq, S₂'_eq, ih_result, p_eq_q]
```

We conclude this chapter by unifying Lemma 4.7 and Lemma 4.10 into a complete formalisation of Theorem 4.6 (Fundamental Theorem of Arithmetic) in Lean.

Proof of Theorem 4.6. By Lemma 4.7 every n can be represented as a product of prime numbers. By Lemma 4.10 this prime factorisation is unique up to the ordering of the factors. □

We introduce the following notation: ∃! — this refers to the unique existence of an element. From the `obtain` tactic we can import from Lemma 4.7 a multiset S with $n = \text{prod}(S)$ and such that every element of S is prime. The `use` tactic is utilised to confirm that S is the set of choice to prove the goal.

```
theorem fundamental_theorem_of_arithmetic (h₁ : n ≥ 2) : ∃! (S :
    Multiset ℕ), (∀ p ∈ S, Nat.Prime p) ∧ n = S.prod := by
  obtain ⟨S, hS⟩ := ftoa_existence h₁
  use S
```

It remains to prove that the set S fulfills the relevant properties, and is the only set with these properties. This is written as a conjunction in Lean which we split with `constructor`. It follows trivially from Lemma 4.7 that the relevant properties are satisfied by S, and by Lemma 4.10, if we introduce a new set S' with the properties, then it must be that $S = S'$.

```
  constructor
  · exact hS
  · intro S' hS'
    exact ftoa_uniqueness h₁ S' S hS' hS
```

Chapter 5

Congruences

In this final chapter we introduce congruences in Lean, and conclude by synthesizing some formalisations developed in each chapter into a final formalisation of Fermat's Little Theorem.

5.1 Introduction to Congruences

Let a, b and n be integers. In Lean we write '`a ≡ b [ZMOD n]`' to refer to the expression $a \equiv b \mod n$. This relationship is equivalent to $a \mod n = b \mod n$, which (using notation introduced in Chapter 2) can be written in Lean as `a % n = b % n` — this is exactly the definition of the congruence relation (identified as `Int.ModEq`) in Mathlib. Note that a, b and n must of integer type — if, for example, they were natural numbers, it would instead be necessary to write `a ≡ b [MOD n]`, unless of course one chooses to cast the natural numbers to integer type. For consistency we do actually perform the latter in this chapter.

Recall an important theorem concerning congruence relations.

Theorem 5.1. *Let a, b and n be integers. Then $a \equiv b \mod n$ if and only if n divides $b - a$.*

Note that the standard convention is that $n \mid a - b$, however in Mathlib this result as well as similar results prefer $n \mid b - a$, therefore we elect to adhere to this approach.

Proof. Suppose first that $a \equiv b \mod n$. Then $a \mod n = b \mod n$, so $(a - b) \mod n = 0 \mod n$. Hence $n \mid a - b$ and so $n \mid b - a$.

Now suppose $n \mid b - a$. Then $n \mid a - b$, so $(a - b) \mod n = 0$. Thus $a \mod n = b \mod n$, and so $a \equiv b \mod n$. □

It is left as an exercise to study the formalisation.

```
theorem congr_iff_dvd : a ≡ b [ZMOD n] ↔ n ∣ b - a := by
  constructor
  · intro h₁
    rw[Int.ModEq] at h₁
    apply Int.emod_eq_emod_iff_emod_sub_eq_zero.mp at h₁
    apply Int.dvd_of_emod_eq_zero at h₁
    exact my.dvd_sub_comm h₁
  · intro h₁
    apply my.dvd_sub_comm at h₁
    apply Int.emod_eq_zero_of_dvd at h₁
    apply Int.emod_eq_emod_iff_emod_sub_eq_zero.mpr at h₁
    rw[← Int.ModEq] at h₁
    exact h₁
```

Now we will apply Theorem 5.1 in a simple result, as well as in a more advanced result. We begin with the simple result — this will be needed to prove Fermat's Little Theorem.

Lemma 5.2. *Let a, b, c and n be integers, and suppose $a \equiv b \mod n$ and $b \equiv c \mod n$. Then $a \equiv c \mod n$.*

Proof. By Theorem 5.1, $n \mid b - a$ and $n \mid c - b$. That is, there exists integers, say x and y, such that $b - a = nx$ and $c - b = ny$ respectively. Therefore $a = b - nx$ and $c = b + ny$ and so $c - a = b + ny - (b - nx) = ny + nx = n(y + x)$. Hence $n \mid c - a$. □

The formalisation is very straightforward, It is similar in style to those encountered in the early stages of chapter 2.

```
lemma mod_trans (h₁ : a ≡ b [ZMOD n]) (h₂ : b ≡ c [ZMOD n]): a ≡ c
    [ZMOD n] := by
  rw[my.congr_iff_dvd] at h₁ h₂
  rcases h₁ with ⟨x, h₁⟩
  rcases h₂ with ⟨y, h₂⟩
  have h₃ : a = b - n * x := by
    linarith
  have h₄ : c = b + n * y := by
    linarith
```

```
  rw[my.congr_iff_dvd]
  rw[h₃, h₄]
  ring_nf
  rw[← mul_add]
  exact my.dvd_mul_right
```

The more 'advanced' result is related to a fundamental congruence result in number theory — Wilson's Theorem. It is an if-and-only-if statement, however the formalisation in Lean of the 'if' is too advanced for this report, and as such we focus only on the 'only if'.

Theorem 5.3. *Let p be a natural number greater than or equal to two. If $(p-1)! \equiv -1 \mod p$, then p is prime.*

Proof. Suppose $(p-1)! \equiv -1 \mod p$. Then by Theorem 5.1, $p \mid -1-(p-1)!$. That is, there is an integer k such that $-1-(p-1)! = pk$.

Suppose for contradiction that p is not prime. Then there exists a natural number, say l, such that $l \mid p$ with $l \geq 2$ and $l < p$. Therefore, $l \mid pk$, so $l \mid -1-(p-1)!$. It is clear that $l \mid (p-1)!$ as $l < p$ and $l > 0$, and since $l \mid -1-(p-1)!$, it follows that $l \mid -1$. Hence $l \mid 1$ thus $l = 1$ (as $l > 0$), however $l \neq 1$ as $l \geq 2$, so we have a contradiction, and conclude that p is prime. □

The formalisation is relatively straightforward. Once we rewrite the congruence relation using Theorem 5.1, the proof becomes an exercise in applying various techniques and results from this paper. The key challenge lies in working with the different variable types, and casting between them.

```
theorem wilsons_theorem_only_if (h₁ : p ≥ 2) (h₂ : (p -
    1).factorial ≡ -1 [ZMOD p]) : Nat.Prime p := by
  rw[my.congr_iff_dvd] at h₂
  rcases h₂ with ⟨k, h₂⟩
  by_contra contra
  rw[Nat.not_prime_iff_exists_dvd_lt h₁] at contra
  rcases contra with ⟨l, h₃, h₄, h₅⟩
  apply Int.ofNat_dvd.mpr at h₃
  apply my.dvd_mul_right_general at h₃
  rw[← h₂, Int.sub_eq_add_neg] at h₃
  have h₆ : l ∣ (p - 1).factorial := Nat.dvd_factorial
    (Nat.zero_lt_of_lt h₄) (Nat.le_sub_one_of_lt h₅)
  apply Int.ofNat_dvd.mpr at h₆
```

```
apply Int.dvd_neg.mpr at h6
have h7 : (l : ℤ) | -1 := (my.dvd_add_iff_dvd_both h3).mpr h6
apply Int.dvd_neg.mp at h7
norm_cast at h7
apply Nat.eq_one_of_dvd_one at h7
apply Nat.ne_of_lt at h4
exact h4 h7.symm
```

5.2 Fermat's Little Theorem

Fermat's Little Theorem has several proofs. Our proof of choice involves results from all previous chapters as well as this final chapter — this renders it an excellent candidate to conclude this report.

Fermat's Little Theorem is the following:

Theorem 5.4. *If p is prime, then for any integer n, if n is not divisible by p, then $n^{p-1} \equiv 1 \mod p$.*

This is equivalent to:

Theorem 5.5. *If p is a prime, then for any integer n, $n^p \equiv n \mod p$.*

We open with a weaker case of Fermat's Little Theorem, which only covers the natural numbers.

Lemma 5.6. *If p is prime, then for any natural number n, $n^p \equiv n \mod p$.*

Proof. We proceed by induction on n. Clearly when $n = 0$, $p \mid 0 - 0^p = 0$, so the proposition is true for $n = 0$. Suppose that the proposition is true for

any n, that is, $n^p \equiv n \mod p$. Then for $n+1$ we have:

$$\begin{aligned}
n+1-(n+1)^p &= 1+n-\sum_{i=0}^{p} 1^{p-i} n^i \binom{p}{i} \quad \text{(by the Binomial Theorem)} \\
&= 1+n-\sum_{i=0}^{p} n^i \binom{p}{i} \\
&= 1+n-\left(\left(\sum_{i=0}^{p-1} n^i \binom{p}{i}\right) + n^p \binom{p}{p}\right) \\
&= 1+n-\left(n^0 \binom{p}{0} + \left(\sum_{i=1}^{p-1} n^i \binom{p}{i}\right) + n^p \binom{p}{p}\right) \\
&= 1+n-\left(1+\left(\sum_{i=1}^{p-1} n^i \binom{p}{i}\right) + n^p\right) \\
&= n-n^p-\sum_{i=1}^{p-1} n^i \binom{p}{i}.
\end{aligned}$$

Now p divides this expression if $p \mid n-n^p$ and $p \mid -\sum_{i=1}^{p-1} n^i \binom{p}{i}$. Clearly $p \mid n-n^p$ by the inductive hypothesis, and $p \mid -\sum_{i=1}^{p-1} n^i \binom{p}{i}$ if $p \mid \sum_{i=1}^{p-1} n^i \binom{p}{i}$, which is true if for all $i = 1, \ldots, p-1$, $p \mid n^i \binom{p}{i}$. By Lemma 4.2 $p \mid \binom{p}{i}$, and hence $p \mid n^i \binom{p}{i}$. Thus $p \mid n-n^p-\sum_{i=1}^{p-1} n^i \binom{p}{i}$. □

First note that, as we are working with natural numbers as well as `ZMOD`, everything is cast to integer type. We begin a proof by induction on n with the `induction'` tactic. Recall that `<;>` (without any specifications) performs a given statement to all goals — we utilise it to rewrite the goals of the base case and inductive step as a divisibility result, and to set up a useful hypothesis, that p is greater than 0. Proceeding with the induction, we first prove that the base case is true. The `ring_nf` tactic removes the 0 and the coercions, leaving the goal to be that p divides -0^p. By `dvd_neg` (formalised in the Appendix) it suffices to prove that p divides 0^p, and by `zero_pow`, since p is prime and thus nonzero (proven by `Nat.ne_zero_of_lt`), we can further reduce the goal to simply that p divides zero, which is true by `Int.dvd_zero`.

```
lemma flt_nat {n p : ℕ} (h₁ : Nat.Prime p) : n ^ p ≡ n [ZMOD p] :=
    by
  induction' n with n ih
  <;> rw [my.congr_iff_dvd]
```

```
<;> have h₂ : p > 0 := Nat.Prime.pos h₁
· ring_nf
  rw [← my.dvd_neg, zero_pow (Nat.ne_zero_of_lt h₂)]
  exact Int.dvd_zero ↑p
```

Moving to the inductive step, we prove that $n + 1 - (n + 1)^p = n - n^p - \sum_{i=1}^{p-1} n^i \binom{p}{i}$ using `calc`. The body of code inside of the `calc` section is self-explanatory — similar ideas can be found throughout Chapter 3.

```
· have : ↑(n + 1) - ↑(n + 1) ^ p = ↑n - ↑n ^ p - Σ i ∈ Finset.Ico
  1 p, ↑n ^ i * choose p i := by
    calc
      ↑n + 1 - (↑n + 1) ^ p = 1 + ↑n - Σ i ∈ Finset.range (p +
  1), 1 ^ (p - i) * ↑n ^ i * choose p i := by
        rw[add_comm, binomial_theorem]
      _ = 1 + ↑n - (Σ i ∈ Finset.range (p + 1), ↑n ^ i * choose p
  i) := by simp
      _ = 1 + ↑n - (Σ i ∈ Finset.range p, ↑n ^ i * choose p i + ↑
  n ^ p * choose p p) := by rw[Finset.sum_range_succ]
      _ = 1 + ↑n - (↑n ^ 0 * choose p 0 + Σ i ∈ Finset.Ico 1 p, ↑
  n ^ i * choose p i + ↑n ^ p * choose p p) := by
        have h₄ : 0 ∉ Finset.Ico 1 p := by simp
        rw[← Nat.Ico_zero_eq_range, (Nat.Ico_insert_succ_left
  h₂).symm, Finset.sum_insert h₄]
      _ = 1 + ↑n - (1 + Σ i ∈ Finset.Ico 1 p, ↑n ^ i * choose p i
  + ↑n ^ p) := by
        simp
        have h₅ : choose p 0 = 1 := by
          unfold choose
          simp
          norm_cast
          exact Nat.div_self (Nat.factorial_pos p)
        have h₆ : choose p p = 1 := by
          unfold choose
          simp
          norm_cast
          apply Nat.div_self (Nat.factorial_pos p)
        rw[h₅, h₆, mul_one]
      _ = ↑n - ↑n ^ p - Σ i ∈ Finset.Ico 1 p, ↑n ^ i * choose p i
  := by
        nth_rw 2 [add_comm]
        rw[sub_add_eq_sub_sub, sub_add_eq_sub_sub]
        ring_nf
```

We substitute our findings into the goal, and use `apply` together with `my.dvd_add` on the goal to generate two new subgoals — that p divides $n - n^p$ and that p divides $-\sum_{i=1}^{p-1} n^i \binom{p}{i}$. The first subgoal is easily proven by rewriting the inductive hypothesis as a divisibility result.

```
rw[this]
apply my.dvd_add
· rw[my.congr_iff_dvd] at ih
  assumption
```

The second subgoal requires more nuance. We first employ `my.dvd_neg` to remove the sign. Next, we pair `apply` with `Finset.dvd_sum`, which reduces the goal to proving that for all $i = 1, \ldots, p-1$, $p \mid n^i \binom{p}{i}$. Note that this result is simply a more general form of Lemma 2.10. We introduce an arbitrary i in `Finset.Ico 1 p`, and deduce from `Finset.mem_Ico` that i is greater than or equal to one and less than p.

```
· rw[← my.dvd_neg]
  apply Finset.dvd_sum
  intro i h7
  rw[Finset.mem_Ico] at h7
```

By Lemma 4.2, p divides $\binom{p}{i}$ if p is an integer prime, and i is greater than zero and less than p. We immediately know by `Nat.prime_iff_prime_int` that since p is a natural prime, it is an integer prime. Further, by reducing i and p back to natural numbers with `norm_cast`, it is trivial to see that i is greater than zero and less than p.

```
  have h8 : (p : ℤ) ∣ choose p i := by
    apply my.prime_dvd_choose (Nat.prime_iff_prime_int.mp h1)
    <;> norm_cast
    · exact h7.left
    · exact h7.right
```

To conclude, since we have that p divides $\binom{p}{i}$, by `dvd_mul_left_general` p divides $n^i \binom{p}{i}$. This completes the formalisation.

```
  exact my.dvd_mul_left_general h8
```

Since we are working towards a proof of Fermat's Little Theorem over all integers, it is convenient to create a separate formalisation proving that Fermat's Little Theorem holds for all numbers of integer type greater than or

equal to zero. Obviously this is equivalent to Lemma 5.6 in mathematical terms, but in Lean we require a change of types.

We redefine n to be an integer (leaving p as a natural number, as powers in Lean cannot be of integer type), and create a new hypothesis in the proposition statement, that n is greater than or equal to zero. We then utilise the `have` tactic together with the Mathlib result '`CanLift.prf`', a result which serves to prove equality between types given certain conditions (in this case that n is non-negative). This proves that there exists a natural number equal to n. We define this number as m via `rcases`. This provides us with a representation of n as a natural number, thus allowing an application of Lemma 5.6.

```
lemma flt_pos_int (h₁ : Nat.Prime p) (h₂ : n ≥ 0): n ^ p ≡ n [ZMOD
    p] := by
  have h₃ : ∃ m : ℕ, m = n := CanLift.prf n h₂
  rcases h₃ with ⟨m, h₃⟩
  rw[← h₃]
  exact flt_nat h₁
```

It remains to prove that Fermat's Little Theorem holds for negative n.

Lemma 5.7. *If p is prime, then for any negative integer n, $n^p \equiv n \mod p$.*

Proof. Define an integer $l = -n$. Clearly $l \geq 0$, therefore by Lemma 5.6 $l^p \equiv l \mod p$.

Since $l = -n$, $n = -l$ so we aim to prove that $(-l)^p \equiv -l \mod p$. That is, $(-1)^p l^p \equiv -l \mod p$. Suppose that p is odd. Then $(-1)^p = -1$ so $(-1)^p l^p = -l^p \equiv -l \mod p$ and hence $l^p \equiv l \mod p$, which we know to be true. Now suppose that p is even. Then $p = 2$ as p is the only even prime, so we know that $l^2 \equiv l \mod 2$. We aim to prove that $(-1)^2 l^2 = l^2 \equiv -l \mod 2$. Clearly $l \equiv -l \mod 2$ as $2 \mid -l - l = -2l$ so by Lemma 5.2, $l^2 \equiv -l \mod 2$. Hence the result holds for all primes p. □

We make use of the `let` tactic to establish an integer l such that $l = -n$. It is easy to see that l is greater than or equal to zero, and so Lemma 5.6 tells us that $l^p \equiv l \mod p$. The `ring_nf` tactic conveniently performs the algebraic manipulation required to obtain that $n = -l$, and substituting this into the goal with `rw`, and further reducing the goal with `neg_pow`, we arrive at a new goal — that $(-1)^p l^p \equiv -l \mod p$.

```
lemma flt_neg_int (h₁ : Nat.Prime p) (h₂ : n < 0) : n ^ p ≡ n [ZMOD
    p] := by
  let l := -n
  have h₃ : l ≥ 0 := by
    ring_nf
    apply Int.neg_nonneg_of_nonpos
    exact Int.le_of_lt h₂
  have h₄ : l ^ p ≡ l [ZMOD p] := flt_pos_int h₁ h₃
  have h₅ : n = -l := by ring_nf
  rw[h₅]
  rw [neg_pow]
```

The next step is to perform the case analysis on p. When p is odd, `Odd.neg_one_pow` reveals that $(-1)^p = -1$, and substituting this into the goal and simplifying, we arrive at $l^p \equiv l \mod p$, which we know to be true.

```
  by_cases case₁ : Odd p
  · have : (-1) ^ p = -1 := Odd.neg_one_pow case₁
    rw[this]
    simp
    exact h₄
```

Applying `Nat.not_odd_iff_even`, it is deduced that the negation of odd is even, and via `two_only_even_prime` (formalised in the Appendix) we find that $p = 2$. We edit the goal and main hypothesis ($\mathtt{h_4}$) to reflect this. Deploying `simp` on the goal reduces the goal to $l^2 \equiv -l \mod p$. We generate a new hypothesis, that $l \equiv -l \mod p$, and close the goal by Lemma 5.2.

```
  · apply Nat.not_odd_iff_even.mp at case₁
    have : p = 2 := (my.two_only_even_prime h₁).mp case₁
    rw[this]
    rw[this] at h₄
    simp
    have h₆ : l ≡ -l [ZMOD 2] := by
      rw[my.congr_iff_dvd, Int.sub_eq_add_neg, ← Int.two_mul]
      exact my.dvd_mul_right
    exact my.mod_trans h₄ h₆
```

The complete formalisations of Lemma 5.6 and Lemma 5.7 are sufficient to prove Theorem 5.5. The `by_cases` tactic is used to generate a check for n greater than or equal to zero and n less than zero, which are exactly the respective lemmas.

```
theorem alternative_flt (h₁ : Nat.Prime p) : n ^ p ≡ n [ZMOD p] :=
    by
  by_cases case : n ≥ 0
  · exact flt_pos_int h₁ case
  · push_neg at case
    exact flt_neg_int h₁ case
```

The primary version of Fermat's Little Theorem, Theorem 5.4, follows quite easily.

Proof of Theorem 5.4. By Theorem 5.5, $n^p \equiv n \mod p$. That is, $p \mid n - n^p = n - n \cdot n^{p-1} = n(1 - n^{p-1})$. By Euclid's Lemma, $p \mid n$ or $p \mid 1 - n^{p-1}$. But by assumption p does not divide n, so $p \mid 1 - n^{p-1}$, and hence $n^{p-1} \equiv 1 \mod p$. □

The formalisation is simple, only requiring some algebraic manipulation followed by an application of Euclid's Lemma. Arriving at the disjunction, we check each case: p cannot divide n so we lead with `exfalso` and show it is impossible, and p dividing $1 - n^{p-1}$ leads directly to the goal.

```
theorem fermats_little_theorem (h₁ : Nat.Prime p) (h₂ : ¬ (p : ℤ) ∣
    n) : n ^ (p - 1) ≡ 1 [ZMOD p] := by
  have h₃ : n ^ p ≡ n [ZMOD p] := alternative_flt h₁
  rw[my.congr_iff_dvd, ← mul_pow_sub_one (Nat.Prime.ne_zero h₁) n]
    at h₃
  nth_rw 1 [← mul_one n] at h₃
  rw[← Int.mul_sub] at h₃
  apply Prime.dvd_or_dvd (Nat.prime_iff_prime_int.mp h₁) at h₃
  obtain h₄ ∣ h₅ := h₃
  · exfalso
    exact h₂ h₄
  · rw[← my.congr_iff_dvd] at h₅
    exact h₅
```

Appendix

This appendix contains formalisations that did not make it into the main text, but may be of interest to the reader. This includes lemmas that were involved in proving results in the main text, as well as some general formalisations based on elementary number theory. Note that no written proofs or code explanations are given.

Lemma A.1. *If a and b are integers, then a divides b if and only if a divides $-b$.*

```
lemma dvd_neg {a b : ℤ} : a ∣ b ↔ a ∣ -b := by
  constructor
  · intro h₁
    rw[← one_mul b]
    rw [Int.neg_mul_eq_neg_mul]
    exact dvd_mul_left_general h₁
  · intro h₁
    rcases h₁ with ⟨x, h₁⟩
    rw [Int.neg_eq_comm] at h₁
    rw[← h₁]
    rw [Int.neg_mul_eq_mul_neg]
    exact dvd_mul_right
```

Lemma A.2. *Let a, b and c be natural numbers. If a divides b and a divides c then a divides $b - c$.*

```
lemma dvd_sub {a b c : ℕ} (h₁ : a ∣ b) (h₂ : a ∣ c) : a ∣ b - c :=
    by
  rcases h₁ with ⟨x, h₁⟩
  rcases h₂ with ⟨y, h₂⟩
  rw[h₁, h₂, ← Nat.mul_sub]
  exact my.dvd_mul_right
```

Lemma A.3. *If a and b are natural numbers, then a divides b and b divides a if and only if $a = b$.*

```
lemma dvd_iff_eq {a b : ℕ} : a ∣ b ∧ b ∣ a ↔ a = b := by
  constructor
  · rintro ⟨h₁, h₂⟩
    rcases h₁ with ⟨x, h₁⟩
    rcases h₂ with ⟨y, h₂⟩
    by_cases case : a = 0
    · rw[case, zero_mul] at h₁
      rw[case, h₁]
    · push_neg at case
      rw[h₁, mul_assoc] at h₂
      nth_rw 1 [← mul_one a] at h₂
      apply (Nat.mul_right_inj case).mp at h₂
      symm at h₂
      apply Nat.eq_one_of_mul_eq_one_right at h₂
      rw[h₂, mul_one] at h₁
      symm at h₁
      exact h₁
  · intro h₁
    rw[h₁]
    exact ⟨my.dvd_reflex, my.dvd_reflex⟩
```

Lemma A.4. *If a and b are integers, then a divides b and b divides a if and only if $a = b$ or $a = -b$.*

```
lemma neg_dvd_reflexive {a : ℤ} : -a ∣ a := by
  use -1
  rw[mul_neg, mul_one, neg_neg]

lemma dvd_reflexive_neg {a : ℤ} : a ∣ -a := by
  use -1
  rw[mul_neg, mul_one]

lemma dvd_iff_eq_or_neg_eq {a b : ℤ} : a ∣ b ∧ b ∣ a ↔ a = b ∨ a =
    -b := by
  constructor
  · rintro ⟨h₁, h₂⟩
    rcases h₁ with ⟨x, h₁⟩
    rcases h₂ with ⟨y, h₂⟩
```

```
    by_cases case : a = 0
    · rw[case, zero_mul] at h₁
      rw[case, h₁]
      left
      rfl
    · push_neg at case
      rw[h₁, mul_assoc] at h₂
      nth_rw 1 [← mul_one a] at h₂
      apply (Int.mul_eq_mul_left_iff case).mp at h₂
      symm at h₂
      apply Int.eq_one_or_neg_one_of_mul_eq_one at h₂
      cases' h₂ with h₃ h₄
      · rw[h₃, mul_one] at h₁
        symm at h₁
        left
        assumption
      · rw[h₄, mul_neg, mul_one] at h₁
        apply Int.eq_neg_comm.mpr at h₁
        right
        assumption
  · intro h₁
    obtain h₂ | h₃ := h₁
    · constructor
      · rw[h₂]
      · rw[h₂]
    · constructor
      · rw[h₃]
        exact my.neg_dvd_reflexive
      · rw[h₃]
        exact my.dvd_reflexive_neg
```

Lemma A.5. *Let a, b and c be integers. If a divides $b - c$ then a divides $c - b$.*

```
lemma dvd_sub_comm {a b c : ℤ} (h₁ : a ∣ b - c) : a ∣ c - b := by
  have : a  -1 * (b - c) := dvd_mul_left_general h₁
  simp at this
  exact this
```

Lemma A.6. *Let a, b and c be integers. If a is nonzero and ab divides ac, then b divides c.*

```
lemma dvd_def {a b c : ℤ} (h₁ : a * b = c) : a ∣ c := by
  rw[← h₁]
  exact my.dvd_mul_right

lemma mul_dvd_mul' {a b c : ℤ} (h₁ : a ≠ 0) (h₂ : a * b ∣ a * c): b
    ∣ c := by
  rcases h₂ with ⟨x, h₂⟩
  rw[mul_assoc] at h₂
  rw[Int.mul_eq_mul_left_iff h₁] at h₂
  exact my.dvd_def h₂.symm
```

Lemma A.7. *Let a, b and c be integers. If a divides $b + c$, then a divides b if and only if a divides c.*

```
lemma dvd_add_iff_dvd_both {a b c : ℤ} (h₁ : a ∣ b + c) : a ∣ b ↔
    a ∣ c := by
  constructor
  · rintro ⟨x, h₂⟩
    rcases h₁ with ⟨y, h₁⟩
    rw[h₂] at h₁
    apply eq_sub_of_add_eq' at h₁
    rw[← mul_sub] at h₁
    use (y - x)
  · rintro ⟨x, h₂⟩
    rcases h₁ with ⟨y, h₁⟩
    rw[h₂, add_comm] at h₁
    apply eq_sub_of_add_eq' at h₁
    rw[← mul_sub] at h₁
    use (y - x)
```

Lemma A.8. *Let a and b be integers. If $a = mb + r$, then d divides a and d divides b if and only if d divides b and d divides r.*

```
lemma dvd_common_divisors {a b d m r : ℤ} (h₁ : a = m * b + r) : d
    ∣ a ∧ d ∣ b ↔ d ∣ b ∧ d ∣ r := by
  constructor
  · rintro ⟨h₂, h₃⟩
    constructor
    · assumption
    · rcases h₂ with ⟨x, h₂⟩
```

```
      rcases h₃ with ⟨y, h₃⟩
      rw[h₂, h₃] at h₁
      have : r = d * (x - m * y) := by
        linarith
      rw[this]
      exact my.dvd_mul_right
  · rintro ⟨h₂, h₃⟩
    constructor
    · rcases h₂ with ⟨x, h₂⟩
      rcases h₃ with ⟨y, h₃⟩
      rw[h₂, h₃] at h₁
      have : a = d * (m * x + y) := by
        linarith
      rw[this]
      exact my.dvd_mul_right
    · assumption
```

Theorem A.9. *The sum of the first n natural numbers is equal to* $\frac{n(n+1)}{2}$.

```
lemma two_dvd_even_mul_odd {n : ℕ} : 2 ∣ (n * (n + 1)) := by
  by_cases h₁ : Even n
  · have h₂ : 2 ∣ n := even_iff_two_dvd.mp h₁
    exact my.dvd_mul_right_general h₂
  · apply Nat.not_even_iff_odd.mp at h₁
    have h₂ : Even (n + 1) := Odd.add_one h₁
    apply even_iff_two_dvd.mp at h₂
    exact my.dvd_mul_left_general h₂

theorem sum_nat {n : ℕ} : ∑ i ∈ Finset.Ico 1 (n + 1), i = n * (n +
    1) / 2 := by
  induction' n with k ih
  · simp
  · have h₁ : 1 ≤ k + 1 := by linarith
    rw[Finset.sum_Ico_succ_top h₁, ih]
    have h₂ : 2 > 0 := by linarith
    apply Nat.mul_left_cancel h₂
    rw[mul_add]
    have h₃ : 2 ∣ k * (k + 1) := my.two_dvd_even_mul_odd
    have h₄ : 2 ∣ (k + 1) * (k + 2) := my.two_dvd_even_mul_odd
    rw[Nat.mul_div_cancel' h₃, Nat.mul_div_cancel' h₄]
    linarith
```

Lemma A.10. *Let n be a natural number. Any two consecutive Fibonacci numbers are coprime.*

```
lemma gcd_fib_coprime {n : ℕ}: ((Nat.fib (n + 1)) : ℤ).gcd
    ((Nat.fib n) : ℤ) = 1 := by
  induction' n with k ih
  · rw [Nat.fib_zero, Nat.fib_one]
    norm_cast
  · rw[Nat.fib_add_two]
    rw[my.gcd_comm]
    nth_rw 2 [add_comm]
    rw[Nat.cast_add]
    nth_rw 2 [← Int.mul_one (Nat.fib (k + 1))]
    rw[← my.gcd_eq_gcd_mul_add]
    exact ih
```

Theorem A.11. *Two is the only even prime.*

```
theorem two_only_even_prime {p : ℕ} (h₁ : Nat.Prime p) : Even p ↔
    p = 2 := by
  constructor
  · intro h₂
    apply Even.exists_two_nsmul at h₂
    rcases h₂ with ⟨k, h₂⟩
    have h₃ : 2 ∣ p := Dvd.intro k h₂.symm
    rw[Nat.dvd_prime h₁] at h₃
    obtain h₄ ∣ h₅ := h₃
    · exfalso
      have : 2 ≠ 1 := by
        exact Nat.succ_succ_ne_one 0
      exact this h₄
    · exact h₅.symm
  · intro h₂
    rw[h₂]
    exact even_two
```

Theorem A.12. *If p is a prime, and if a is an integer, then if p divides a, then p does not divide $a + 1$.*

```
lemma prime_not_dvd_one {p : ℤ} (h₁ : Prime p) : ¬ p ∣ 1 := by
  by_contra h₂
  rcases h₂ with ⟨k, h₂⟩
  have h₃ : IsUnit p := isUnit_of_mul_eq_one p k h₂.symm
  have h₄ : ¬ IsUnit p := Prime.not_unit h₁
  exact h₄ h₃

theorem prime_not_dvd_succ {a p : ℤ} (h₁ : Prime p) (h₂ : p ∣ a) : ¬
    p ∣ (a + 1) := by
  rcases h₂ with ⟨k, h₂⟩
  by_contra h₃
  rcases h₃ with ⟨l, h₃⟩
  have h₄ : 1 = p * (l - k) := by
    rw[mul_sub, ← h₂, ← h₃]
    linarith
  symm at h₄
  apply Dvd.intro (l - k) at h₄
  have h₅ : ¬ p ∣ 1 := my.prime_not_dvd_one h₁
  exact h₅ h₄
```

References

[1] H. Geuvers. Proof assistants: History, ideas and future. *Sadhana (Bangalore)*, 34(1):3–25, 2009.

[2] Freek Riedijk. Formalizing 100 theorems. `https://www.cs.ru.nl/~freek/100/`. Accessed: 2024-11-18.

[3] About lean. `https://lean-lang.org/about/`. Accessed: 2025-04-26.

[4] Lean (proof assistant). `https://en.wikipedia.org/wiki/Lean_(proof_assistant)`. Accessed: 2025-04-26.

[5] The mathlib Community. The lean mathematical library. 2019.

[6] Loogle! `https://loogle.lean-lang.org/`. Accessed: 2025-04-26.

[7] Moogle. `https://www.moogle.ai/`. Accessed: 2025-04-26.

[8] Missing undergraduate mathematics in mathlib. `https://leanprover-community.github.io/undergrad_todo.html`. Accessed: 2025-04-26.

[9] Gareth A. Jones and J. Mary (Josephine Mary) Jones. *Elementary number theory / Gareth A. Jones and J. Mary Jones.* Springer Undergraduate Mathematics Series. Springer, London, England, 1998 - 1998.

[10] Gove W. Effinger and Gary L. Mullen. *Elementary number theory Gove Effinger, Gary L. Mullen.* Chapman Hall/CRC, Boca Raton, 2021.

[11] D.M. Burton. *Elementary Number Theory.* McGraw-Hill, 2010.

[12] Marianne Johnson and Daniel Loughran. MATH32072: Number Theory, 2024. Lecture Notes, University of Manchester.

[13] Jeremy Avigad and Patrick Massot. Mathematics in lean, 2024.

[14] Natural Number Game. https://adam.math.hhu.de/#/g/leanprover-community/nng4. Accessed: 2025-04-26.

[15] Introduction to Lean Using Number Theory: Complete Code. https://github.com/LukeSmith4/intro-to-lean.

[16] Heather Macbeth. The mechanics of proof. https://hrmacbeth.github.io/math2001/06_Induction.html. Accessed: 2025-03-29.